



โครงสร้างข้อมูลรายการโยง (Linked-Lists Data Structure)

โดย ผศ.ดร.เชาวลิต ชั่นคำ

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยรามคำแหง

- 1. มีความรู้ สามารถอธิบายแนวคิดและความหมายของโครงสร้างรายการโยงได้
- 2. มีความรู้และอธิบายประเภทของรายการโยงได้
- **3.** มีความรู้และสามารถสร้างและการทอ้งรายการโยงทางเดียวและรายการโยงแบบสองทาง
- 4. มีความรู้และสามารถใช้ตัวดำเนินการ เพิ่ม แทรก ลบ และค้นหาข้อมูลในรายการโยงแบบ ทางเดียว
- 5. มีความรู้และสามารถอธิบายแนวคิดและรูปแบบของรายการโยงแบบสองทาง
- **6.** มีความรู้และสามารถใช้ตัวดำเนินการ เพิ่ม แทรก ลบ และค้นหาข้อมูลในรายการโยงแบบทางสองทาง
- **7.** มีความรู้และสามารถอธิบายแนวทางการประยุกต์ใช้โครงสร้างข้อมูลรายการโยงโดยใช้ภาษาจาวา

- เนื่องจากโครงสร้างข้อมูลแบบอาร์เรย์ยังไม่ยืดหยุ่นพอ โดยเฉพาะในการเพิ่มและลบข้อมูลในโครงสร้างข้อมูล อีกทั้งการบริหารจัดการเนื้อที่ที่ยังไม่มีประสิทธิภาพเท่าที่ควร ในขณะที่โครงสร้างข้อมูลแบบกองซ้อนและแถวคอย มีหลักการทำงานเฉพาะที่เหมาะสมสำหรับข้อมูล ที่มีคุณลักษณะตรงกับรูปแบบของแต่ละโครงสร้างเท่านั้น ดังนั้น จึงจำเป็นต้องแสวงหาโครงสร้างข้อมูล ที่มีความยืดหยุ่นมากขึ้น สนับสนุนการเพิ่มและการลบข้อมูลได้อย่างสะดวกรวดเร็วมากยิ่งขึ้น รายการโยง หรือ ลิงค์ลิสต์จึงถูกนำมาใช้งาน เนื้อหาบทนี้นำเสนอ ความหมาย ประเภท การสร้างโครงสร้างข้อมูล การเพิ่ม การเชื่อมโยง การแทรกข้อมูล และการลบข้อมูล ของรายการโยงทางเดียว การค้นหาข้อมูลในรายการโยงทางเดียว การสร้างรายการโยงและดำเนินการกับรายการโยงแบบสองทาง รวมถึงแนวทางการบูรณาการและประยุกต์ใช้งานรายการโยงอีกด้วย

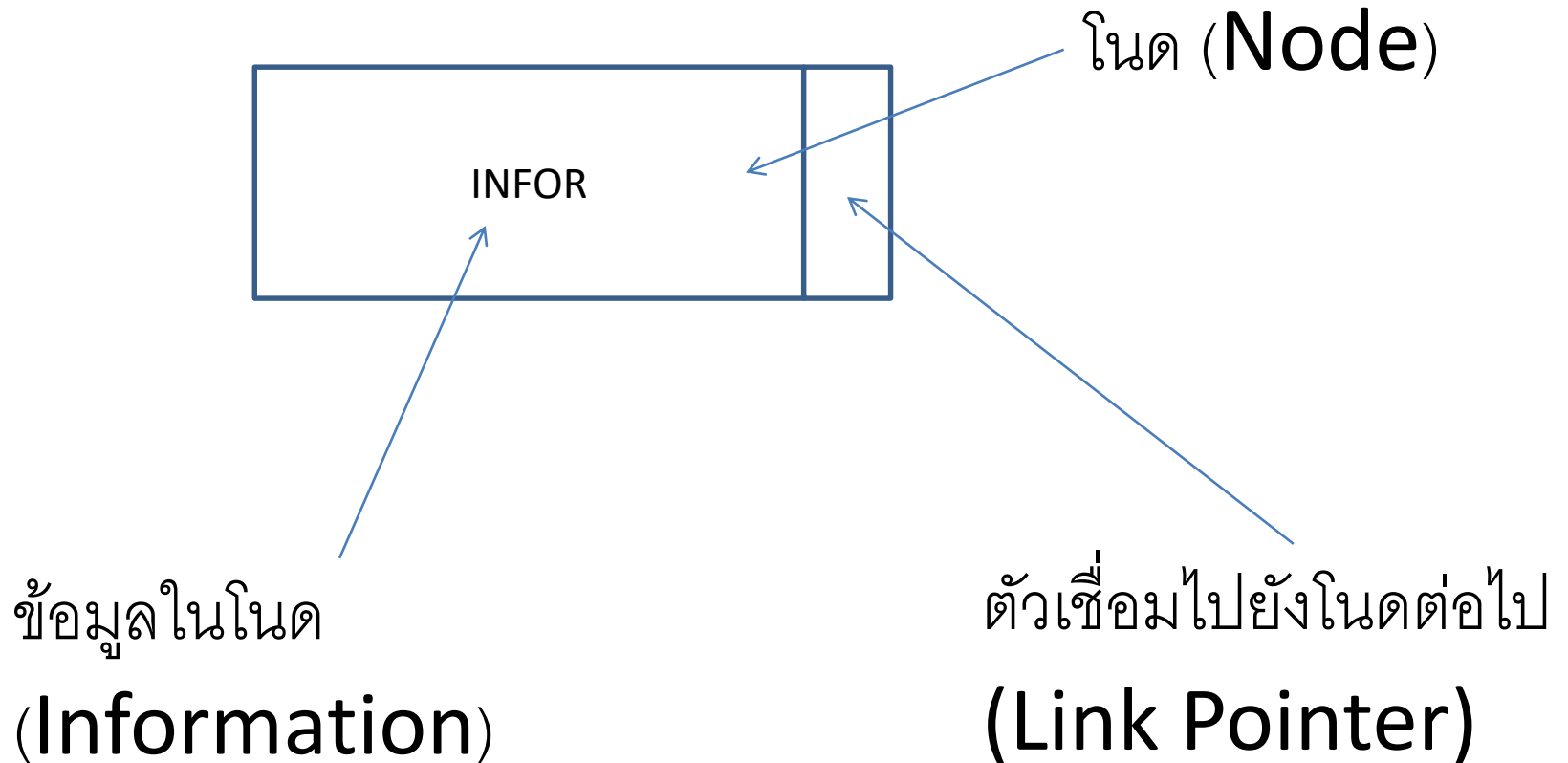
- **รายการโยง** คือ โครงสร้างข้อมูลที่ข้อมูลแต่ละรายการโยงกันต่อเนื่องกันไปจากจุดเริ่มต้นเชื่อมโยงข้อมูลรายการแรกหลังจากนั้นจะโยงต่อกันไปแบบรายการต่อรายการ ไปจนกระทั่งถึงรายการสุดท้าย
- ข้อมูลแต่ละรายการเรียกว่า โหนด (**Node**) การเชื่อมโยงต่อกันใช้ตัวเชื่อมโยงเรียกว่า ลิงค์ (**Link**) การเชื่อมโยงสามารถเชื่อมโยงได้ทั้งทางเดียวและสองทาง โครงสร้างข้อมูลชนิดนี้ มีจุดเด่น คือ สามารถเพิ่ม หรือลบข้อมูลแต่ละโหนดในโครงสร้างได้โดยง่าย

รายการโยง (Link Lists)

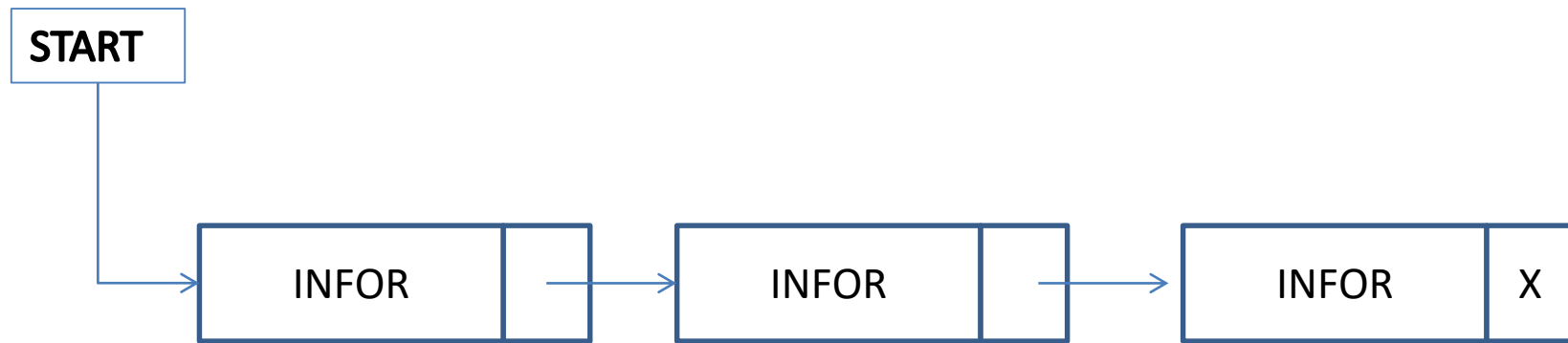
- เป็นข้อมูลแบบรายการต่อเนื่อง
- เชื่อมโยงจากรายการที่หนึ่ง ไปจนถึงรายการสุดท้าย
- ข้อมูลแต่ละรายการเรียกว่า โหนด (Node)
- การเชื่อมโยงต่อกันใช้ตัวเชื่อมโยง ลิงค์ (Link)
- การเชื่อมโยงสามารถเชื่อมโยงได้ทั้งทางเดียว และสองทาง
- ข้อดี สามารถเพิ่ม ลบ ข้อมูลแต่ละรายการได้โดยง่าย

- นิยามที่ **6.1** กำหนดให้ **Node** คือ ข้อมูลรายการรายการโยง 1 รายการ ที่ประกอบด้วย ส่วนข้อมูลและส่วนของการเชื่อมโดย โดยที่ **INFOR** คือ ข้อมูลที่บรรจุใน **LINK** คือ ตัวชี้สำหรับเชื่อมโยงไปยัง **Node** ต่อไป **START** คือ ตัวชี้ไปยังโนดแรกของโครงสร้างรายการโยง

มุมมองลิงก์ลิสต์

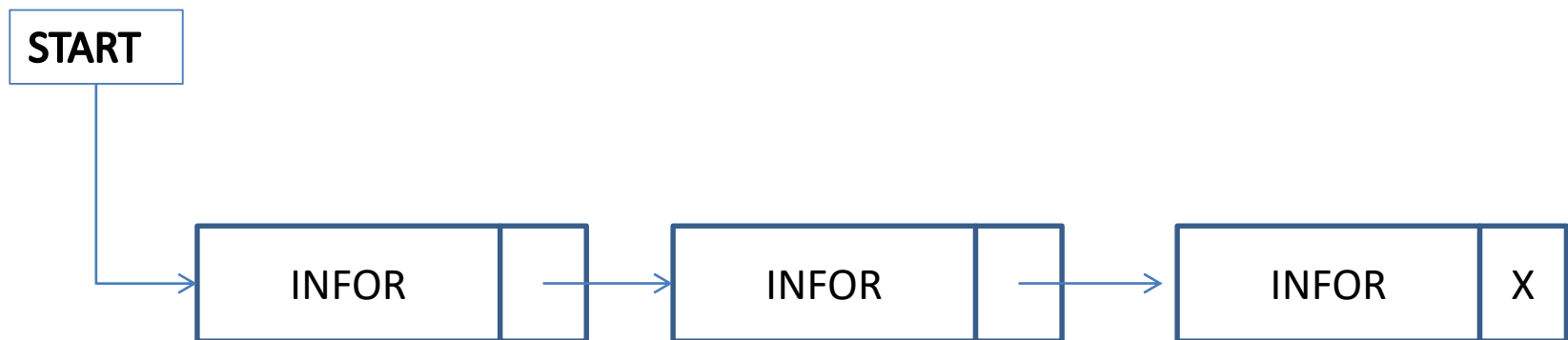


มุมมองลิงค์ลิสต์

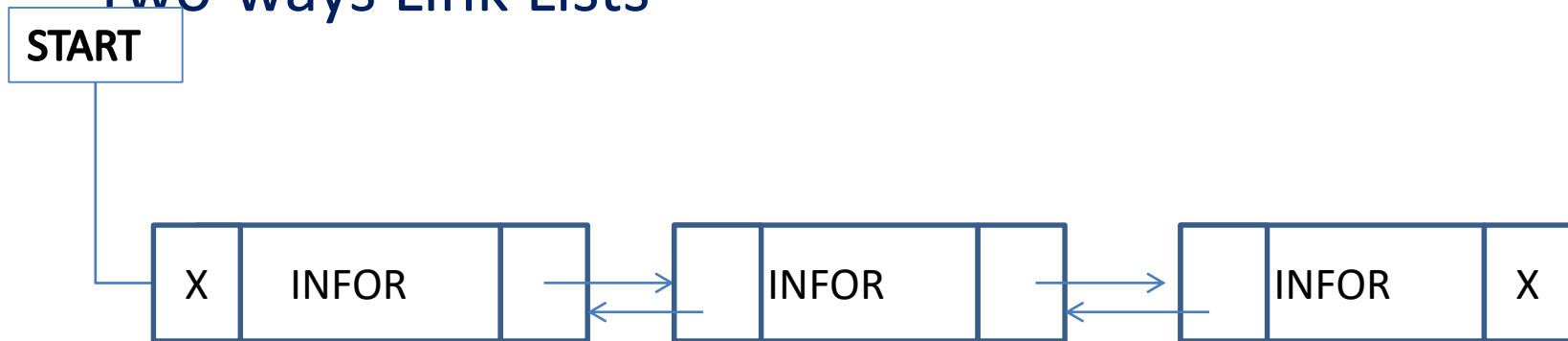


ประเภทของลิ่งค์ลิสต์

- One-way Link Lists



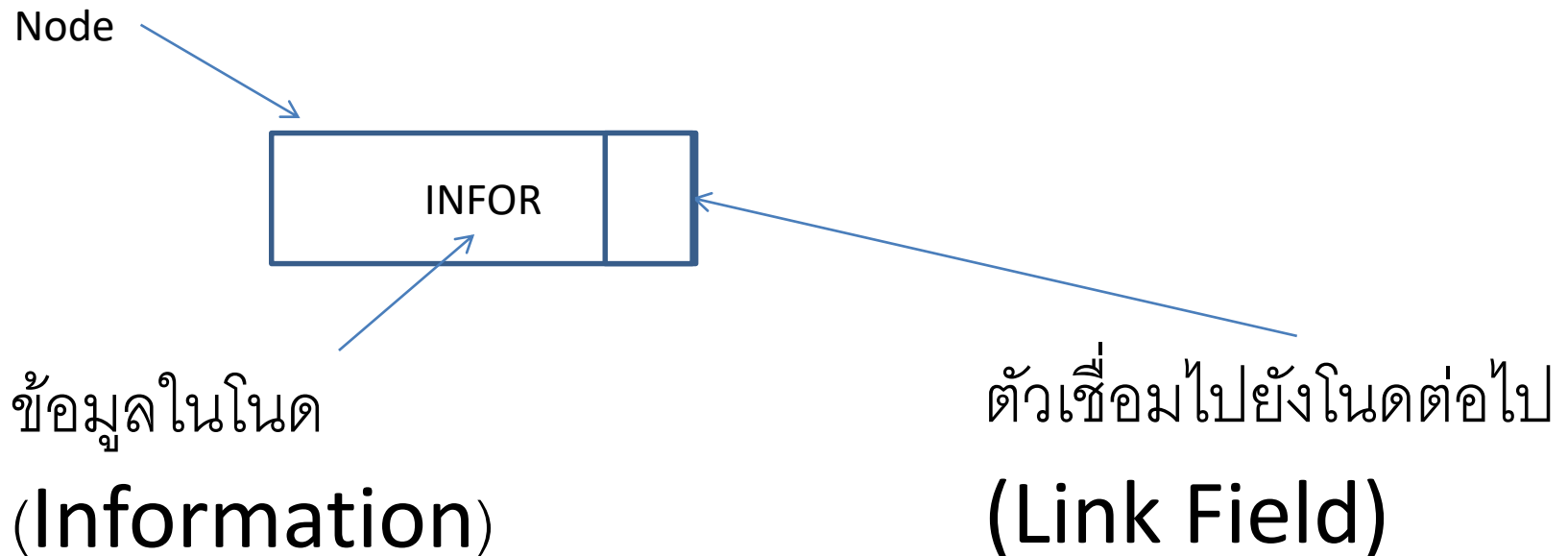
- Two-ways Link Lists



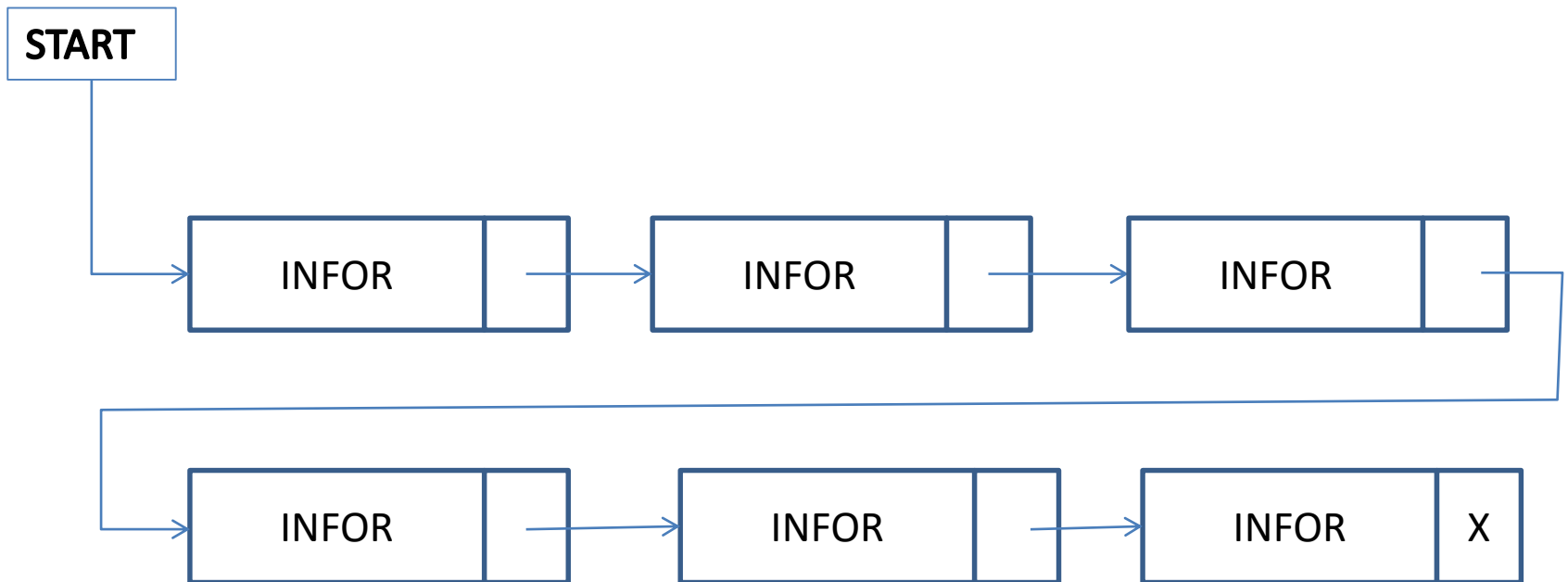
ลิงค์ลิสต์ทางเดียว(One-way Link Lists)

- เรียกแต่ละรายการข้อมูลว่า **Node** ประกอบด้วย 2 ส่วนคือ
 - ส่วนข้อมูล เรียกว่า **Information** ของแต่ละอีลิเมนต์
 - ส่วนลิงค์ฟิลด์ (**Link Field**)

Node

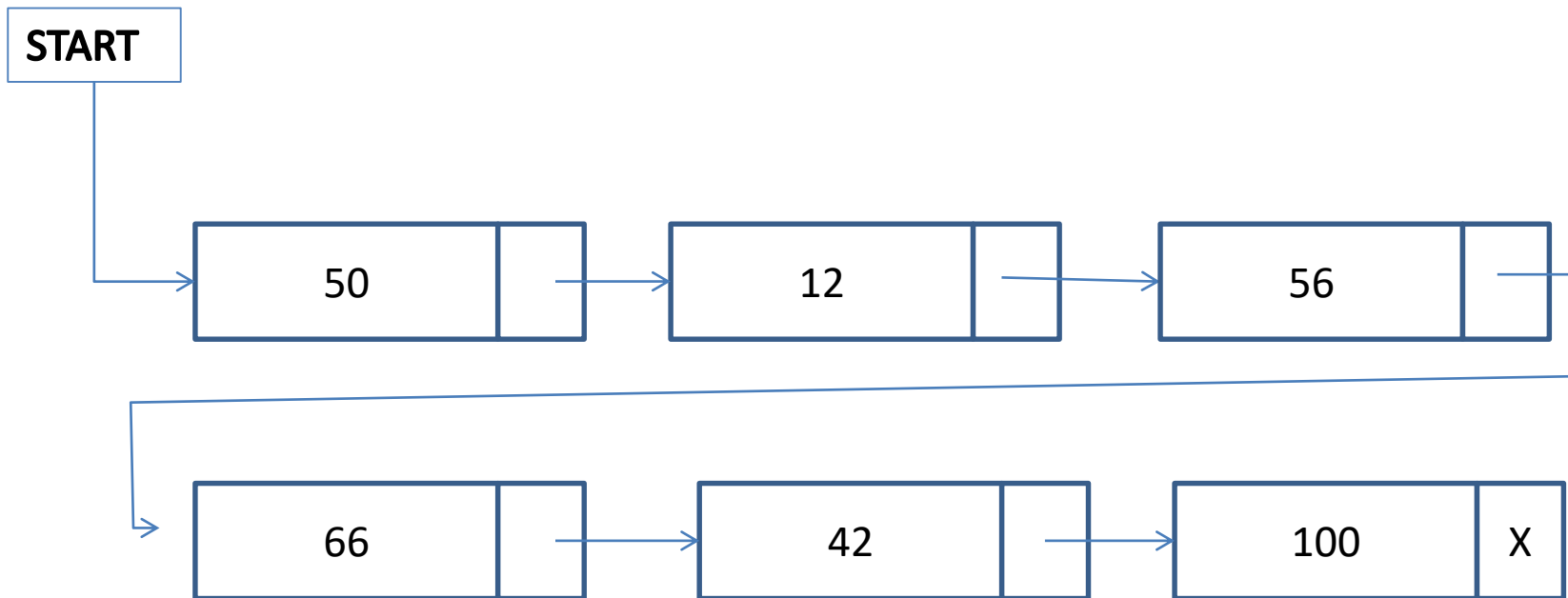


คุณลักษณะทั่วไป



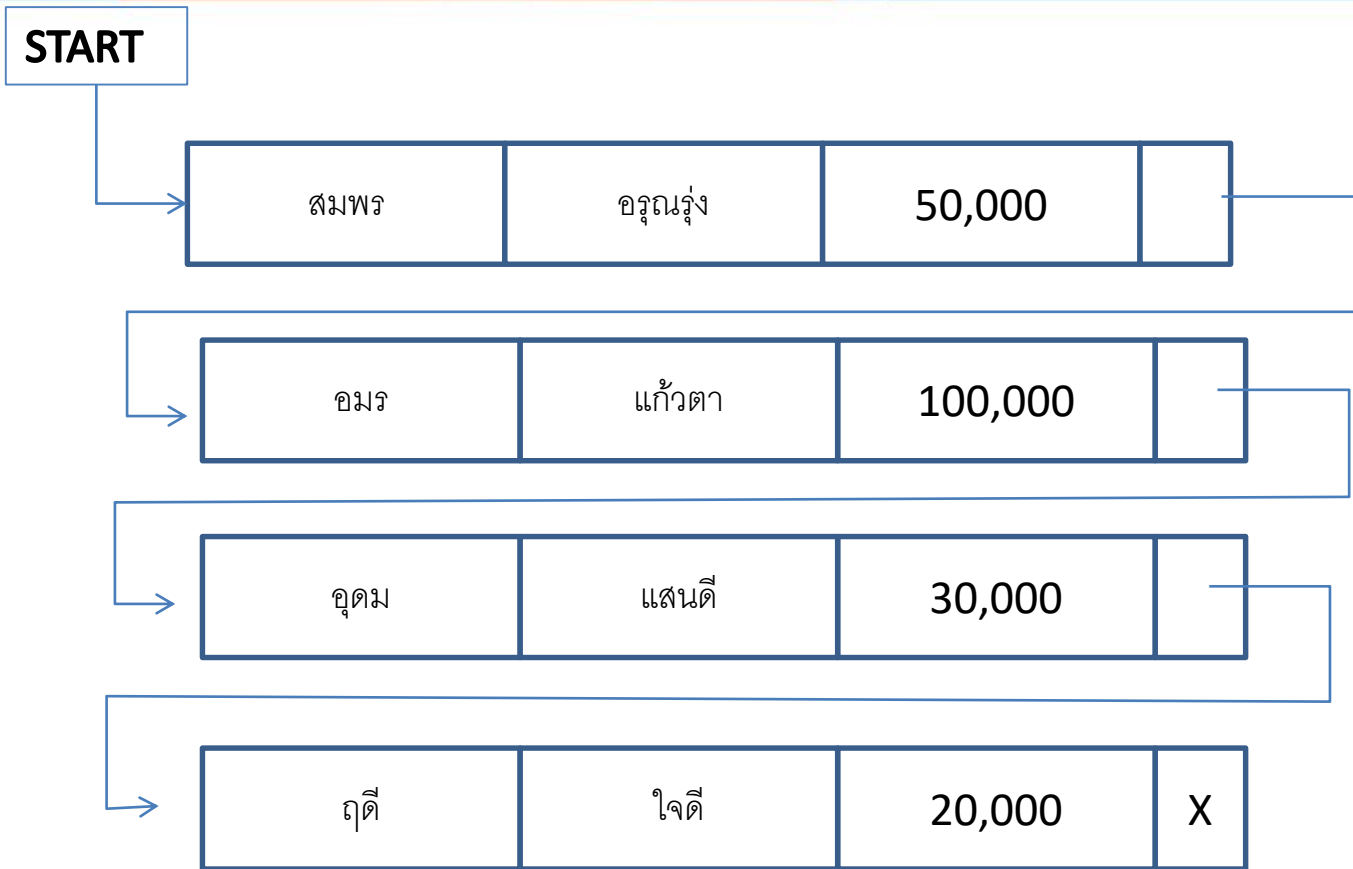
X หมายถึง NULL

ตัวอย่างการเก็บข้อมูล



X หมายถึง NULL

ตัวอย่างการเก็บข้อมูล



X หมายถึง NULL

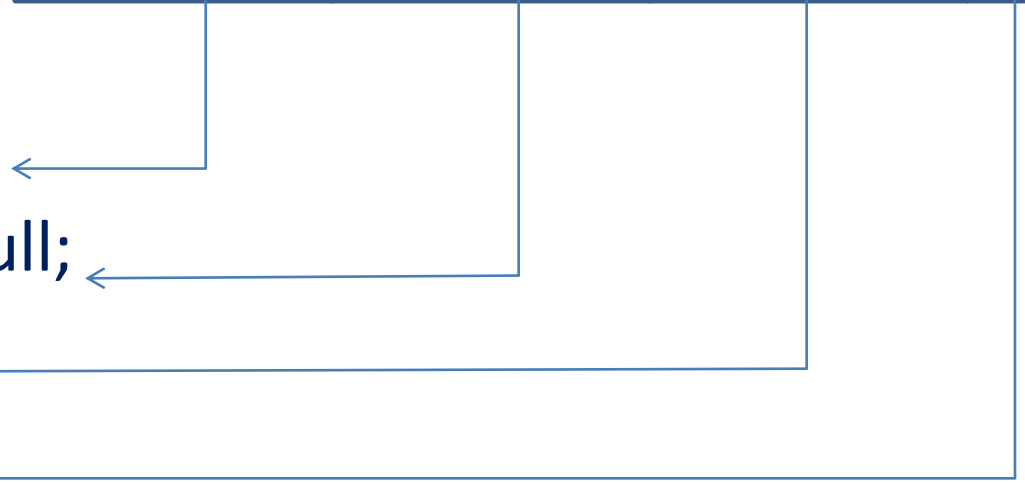
แนวทางอิมพลีเมนต์

- public class Node {
- Object INFOR;
- Node LINK=null;
- }



- public class Node {
- String name=null;
- String surname=null;
- int salary=null;
- Node LINK=null;
- }

name	surname	salary	X
------	---------	--------	---



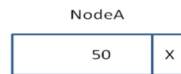
ตัวอย่างการสร้างโนดและบรรจุข้อมูล

- การสร้างโนดด้วยภาษาจาวา
 - `Node NodeA = new Node();`
- บรรจุข้อมูล
 - `NodeA.INFOR = 50;`

NodeA

50	X
----	---

เมื่อดำเนินการทั้งการสร้างและบรรจุข้อมูลแล้ว จะได้โน้ตตามแนวคิดดังกล่าวดังรูปที่ 6.5



รูปที่ 6.5 แสดงข้อมูลในโน้ต A

ตัวอย่างแนวคิดการสร้างรายการโยงและการเชื่อมโยงข้อมูลที่ละขั้นตอน เริ่มจากการสร้างโน้ต บรรจุข้อมูล และทำการเชื่อมโยง โดยให้ START เป็นจุดเริ่มต้นของรายการโยง อธิบายพอสังเขปดังนี้

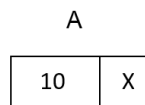
(1) สร้างโน้ตจากโครงสร้างที่กำหนดไว้ให้ Node A = New Node(); Node B = New Node(); จะได้โน้ตขึ้นมาดังนี้



รูปที่ 6.6 แสดงโน้ต A และ B เมื่อถูกสร้างขึ้น

(2) บรรจุข้อมูลลงไป โดยระบุ

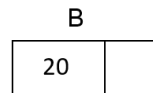
A.INFOR = 10



รูปที่ 6.7 แสดงโน้ต A เมื่อบรรจุข้อมูล

(3) บรรจุข้อมูลลงไปโดยระบุ

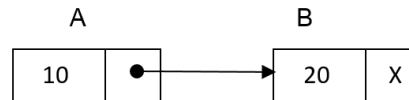
B.INFOR = 20



รูปที่ 6.8 แสดงโนด B เมื่อบรรจุข้อมูล

(4) เชื่อมโยงข้อมูล A->B โดยระบุ

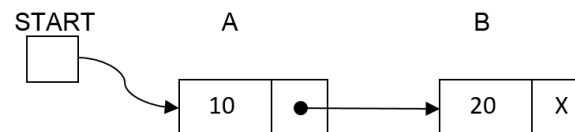
A.LINK = B



รูปที่ 6.9 แสดงโนด A เชื่อมไปยังโนด B

(5) กำหนด START ขึ้นไปยังโนดแรก โดยระบุ

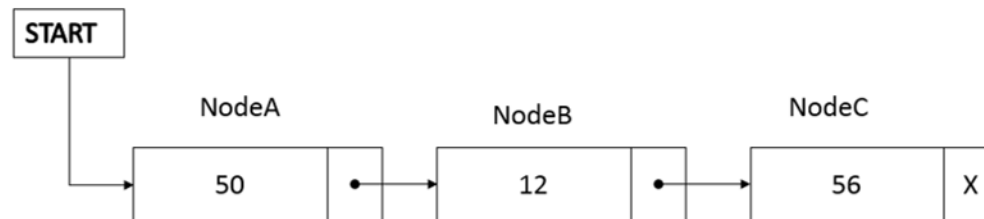
START = A



รูปที่ 6.10 แสดง START เชื่อมไปยังโนด A

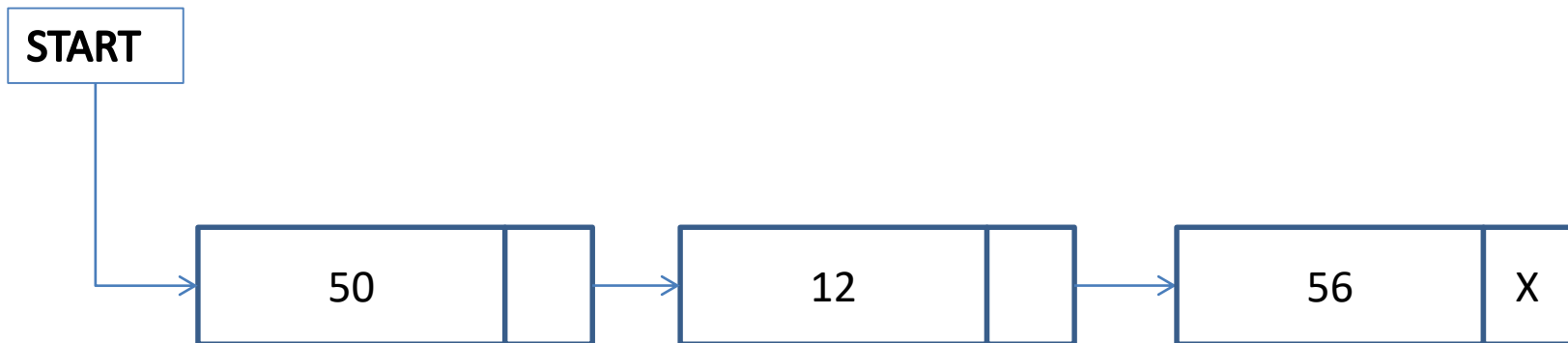
ตัวอย่างที่ 6.1 จงวาดภาพและแสดงการสร้างโนด จาก NodeA, NodeB และ NodeC นำข้อมูล
เข้า 50, 12 และ 56 ตามลำดับ จากนั้นดำเนินการเชื่อมโยงข้อมูลโดย กำหนดให้ START ชี้ไป
ยัง NodeA เริ่มต้น จากนั้น NodeA-> NodeB->NodeC

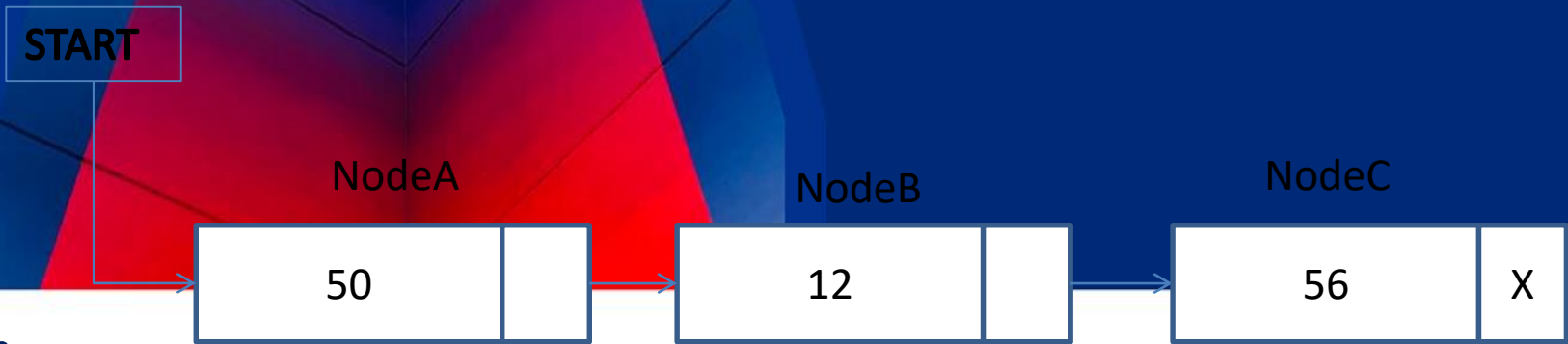
เฉลย: สามารถวาดภาพลิงค์ลิต์ดังกล่าวได้ดังรูปที่ 6.11



รูปที่ 6.11 แสดงการเชื่อมโยงกันของโนดแบบมีลิงค์ทางเดียว โดยกำหนดชื่อของโนดอย่างง่าย

การสร้างเป็นลิงก์ลิสต์





- Node START;
- Node NodeA = new Node();
- NodeA.INFOR = 50;
- Node NodeB = new Node();
- NodeB.INFOR = 12;
- Node NodeC = new Node();
- NodeC.INFOR = 56;
- START.LINK = NodeA;
- NodeA.LINK = NodeB;
- NodeB.LINK=NodeC

การดำเนินการกับลิสต์

- การทอ้ง
- การเพิ่ม
- การลบ
- การค้นหา

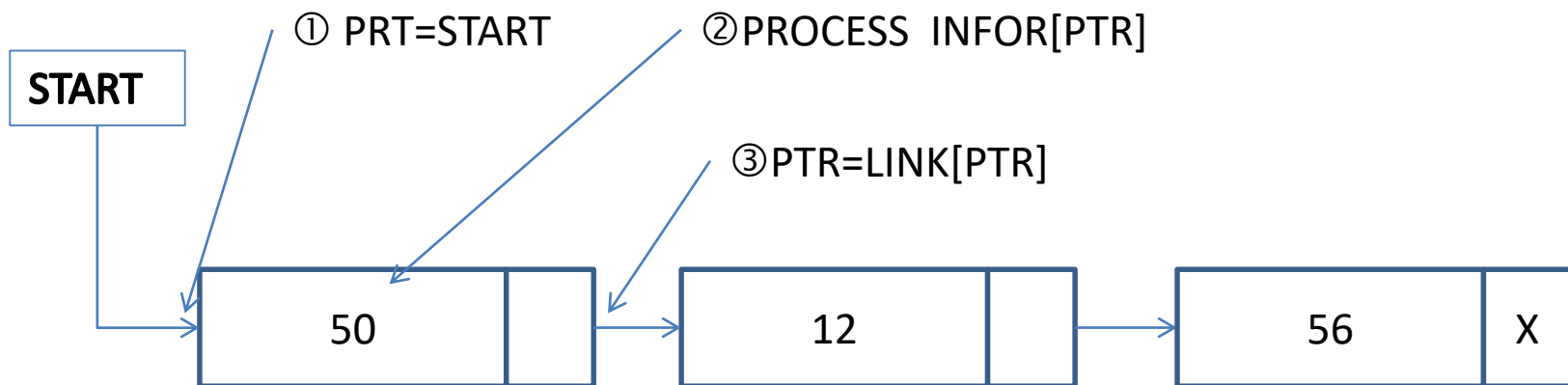
การท่องเที่ยวแบบทางเดียว

- เนื่องจากโครงสร้างรายการโยงเป็นข้อมูลที่อยู่กันอย่างต่อเนื่องเชื่อมโยงจากรายแรกไปยังรายการสุดท้ายโดยอาศัยตัวชี้ **LINK** ในการเชื่อมโยง โดยแนวความคิดการท่องเที่ยวโครงสร้างจะต้องเริ่มเข้าที่โนดแรกสุด จากตำแหน่งที่ **START** ชี้อยู่ แล้วดำเนินการกับข้อมูลที่ตำแหน่งนั้น จากนั้นก็ตามตัวชี้ **LINK** ต่อเนื่องกันไป ดังนั้นเพื่อความเข้าใจในการใช้ตัวแปรสำหรับเชื่อมโยง
- **นิยามที่ 6.2** กำหนดให้ **PTR** คือ ตัวชี้ที่ใช้ชี้ไปยังโนดของรายการโยงใดๆ

การท่องลึ่กัลิสต์(Traversing a Linked Lists)

- เริ่มต้นด้วยการนำเอาพอยเตอร์ PTR ไปชี้ที่ START
- แล้วตามเข้าไปดำเนินการกับ INFOR ในแต่ละโนด
- หลังจากนั้นตามลึ่ก LINK ของแต่ละโนดไป จนกระทั่ง LINK เป็น NULL

การท่องลิสต์



ดำเนินการ ② ③ จนกระทั่ง **LINK[PRT]=NULL**

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. นำตัวชี้ไปยัง START เพื่อกำหนดโนดแรกที่จะท่อง
2. ดำเนินการกับข้อมูล INFOR ของโนดที่ PTR ชี้อยู่ INFOR [PTR]
3. ขยับ PTR โดยกำหนด $PTR = LINK.PTR$ เป็นการขยับตัวชี้ไปยังโนดถัดไป
4. หาก $PTR \neq NULL$ วนดำเนินการตามขั้นตอนที่ 2
5. หาก $PTR = NULL$ จบการทำงาน

Algorithm 6.1: TraversingOneWay

Input : LIST, PTR, START

Output : All nodes in LIST are traversed

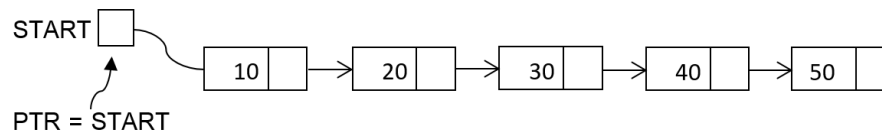
1. $PTR := START$
2. While $PTR \neq NULL$ Do
3. Process INFOR[PTR]
4. $PTR := LINK[PTR]$
5. End While
6. Return

ตัวอย่างที่ 6.2 กำหนดรายการโยง ดังนี้ 10, 20, 30, 40, 50 จงอธิบายการท่องตามทึ่อัลกอริทึม

6.1

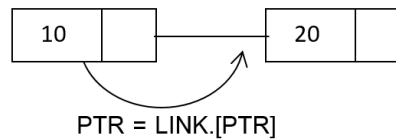
แนวทางอธิบาย

จากโจทย์ สามารถวาดเป็นภาพของรายการโยงได้ดังนี้
เบื้องต้นต้องกำหนดตัวแปรดังนี้



รูปที่ 6.13 แสดงภาพรายการโยงเริ่มต้นของตัวอย่างที่ 6.2

1. PROCESS INFOR.PTR ประมวลผลข้อมูลที่ PTR ชี้อู่
ในตัวอย่างนี้คือ ประมวลผล 10 นั้นเอง
2. PTR = LINK.PTR เป็นการเลื่อนตามตัวชี้ถัดไปที่โนดแรกชื้ออยู่



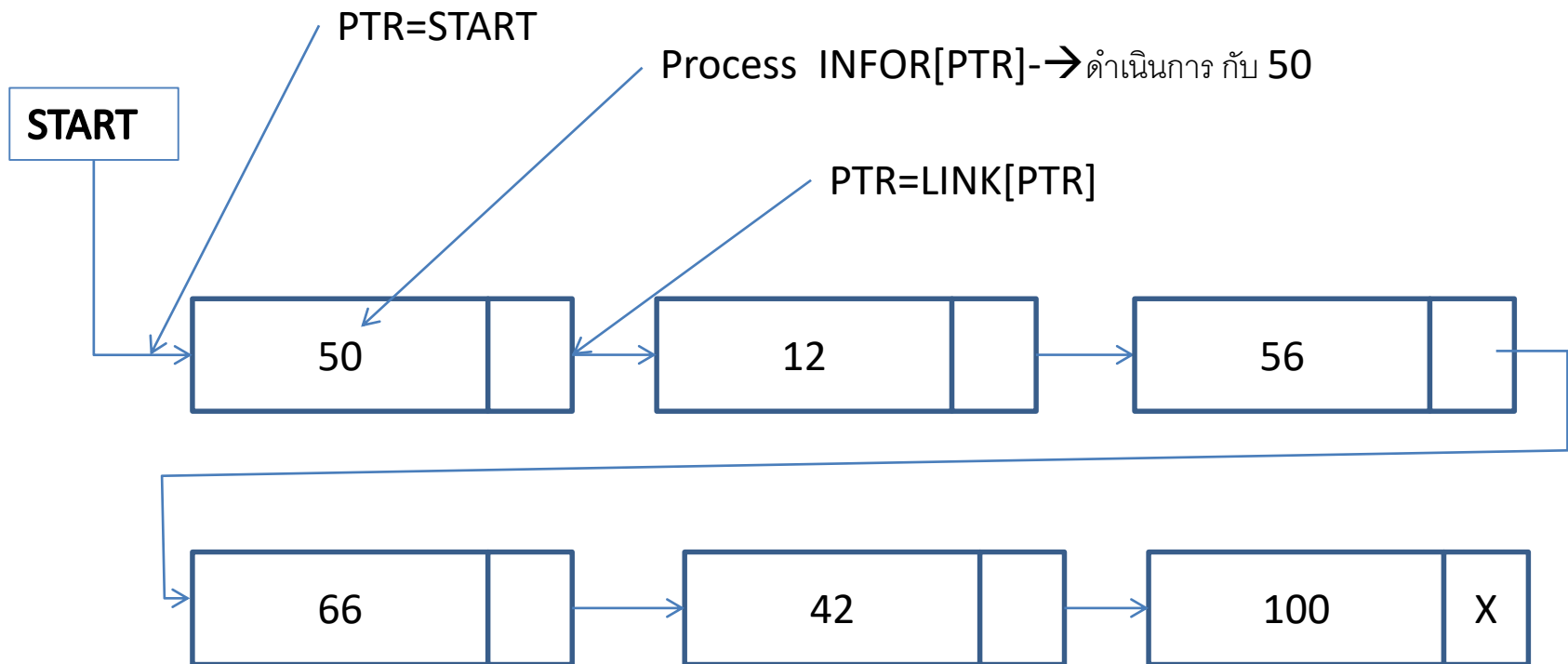
รูปที่ 6.14 แสดงการเลื่อนตัวชี้ไปยังโนดถัดไป

3. ตรวจสอบว่า $PTR \neq NULL$ ดำเนินการต่อไป



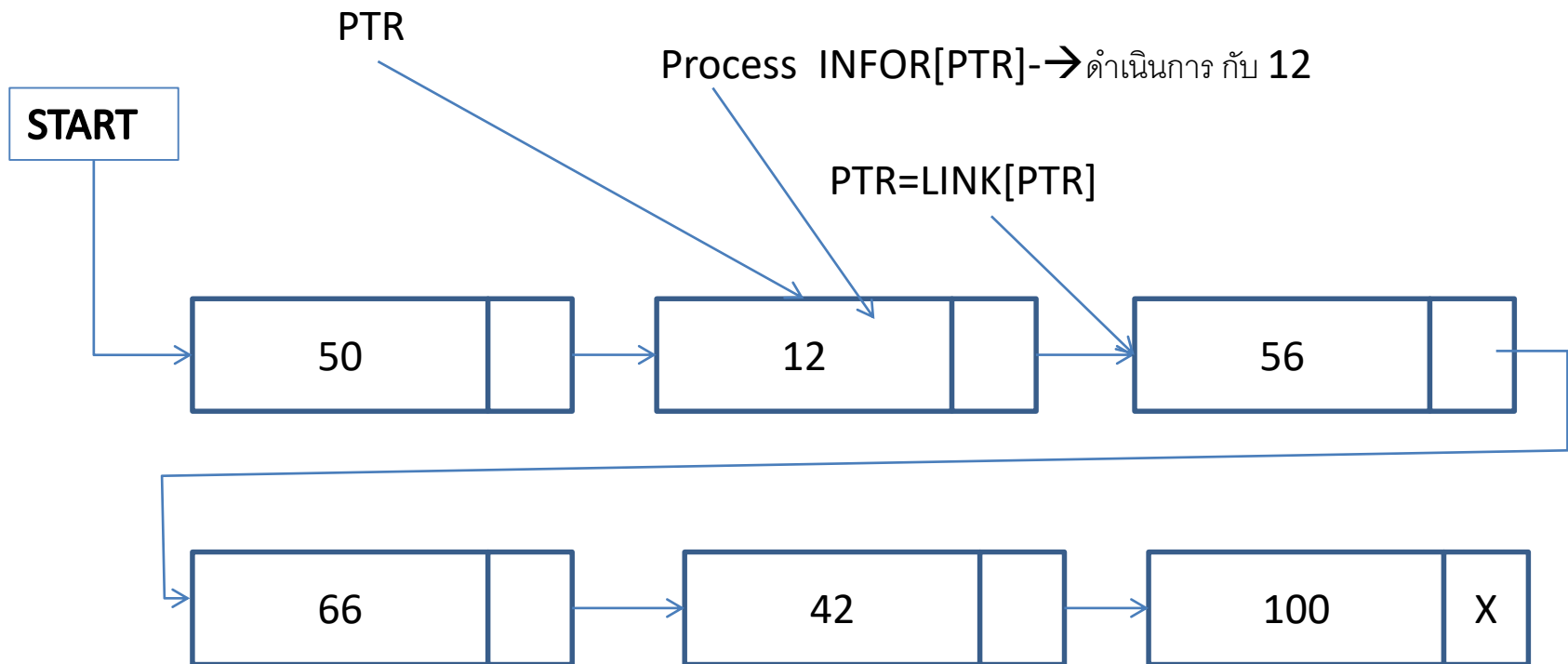
ตัวอย่างที่ 6.3 จงแสดงการท่องจากรายการโยง โดยแสดงประกอบภาพการทำงานของ
อัลกอริทึม มีข้อมูล 50, 12, 56, 66, 42, 100

ตัวอย่างการท่องलि่ค้ลิสต์ตามอัลกอริทึม



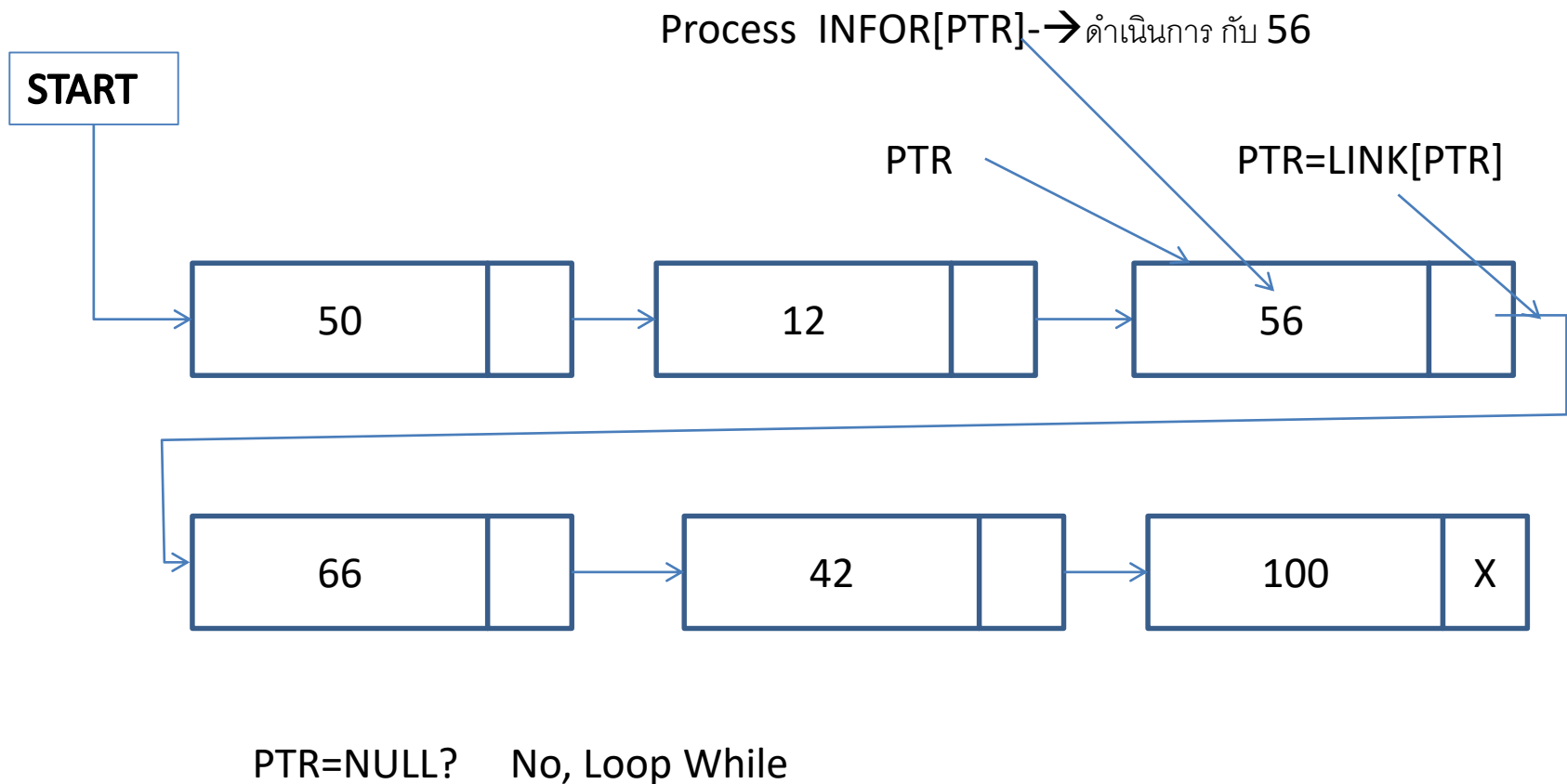
PTR=NULL? No, Loop While

ตัวอย่างการท่องलिค์ลิสต์ตามอัลกอริทึม

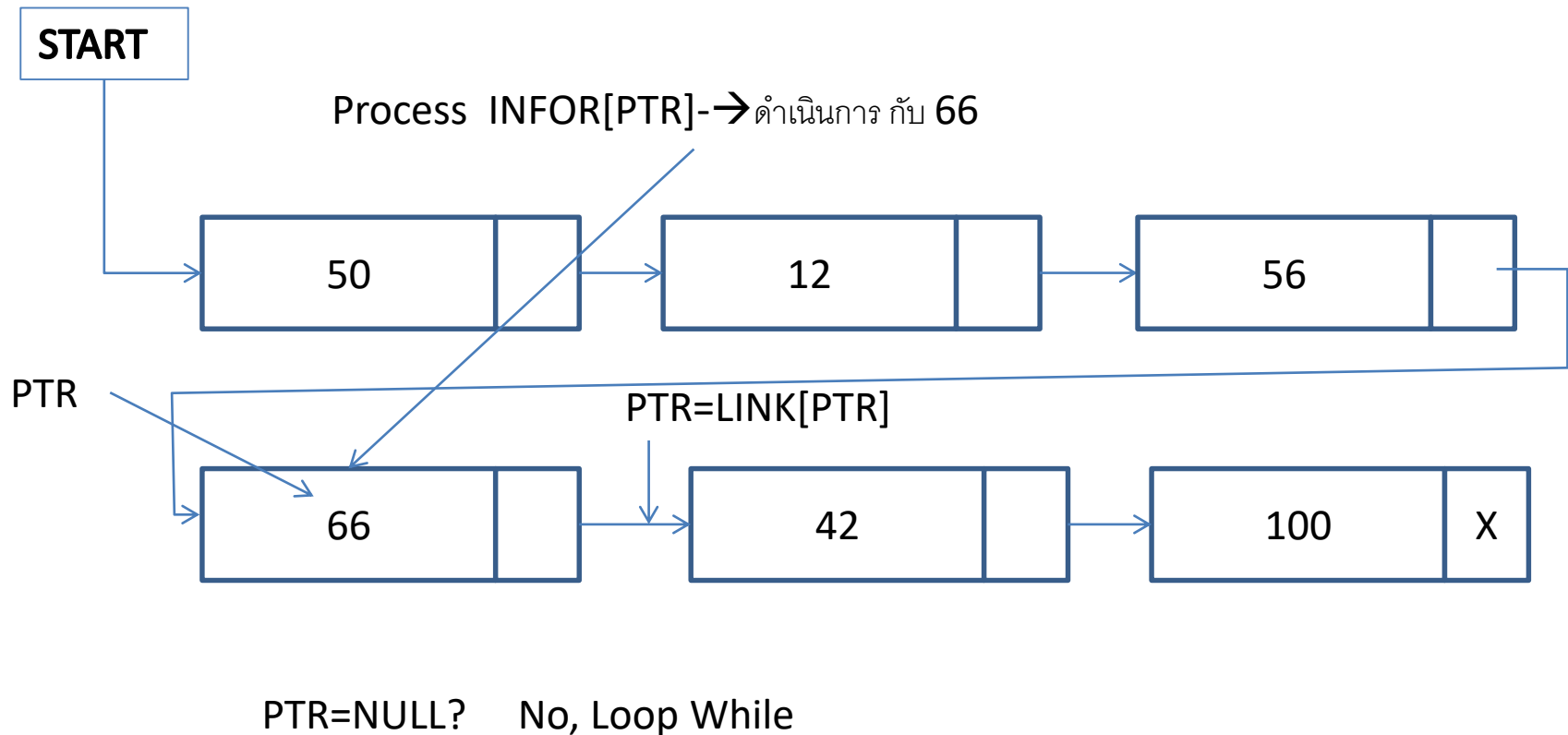


PTR=NULL? No, Loop While

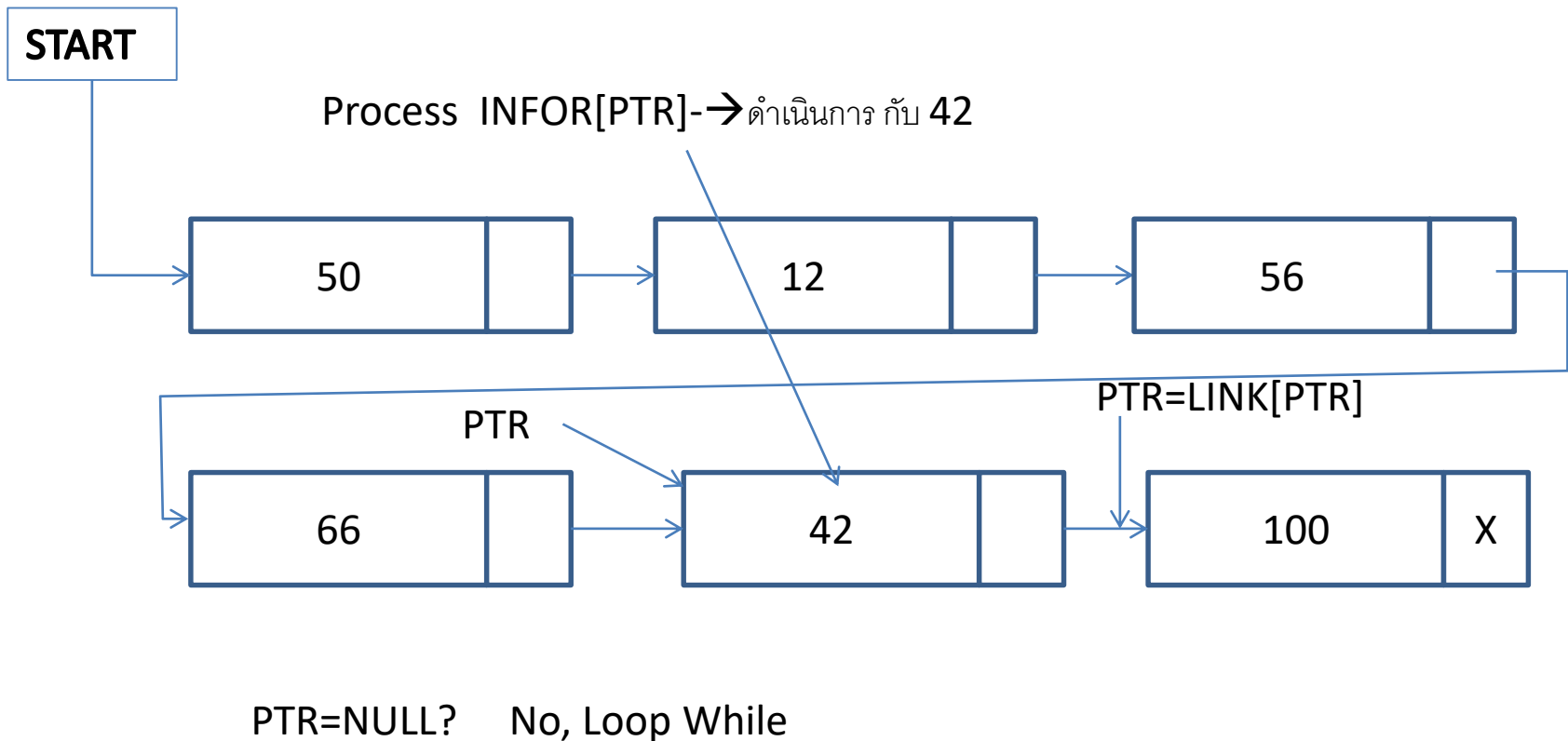
ตัวอย่างการท่องलि่ค้ลิสต์ตามอัลกอริทึม



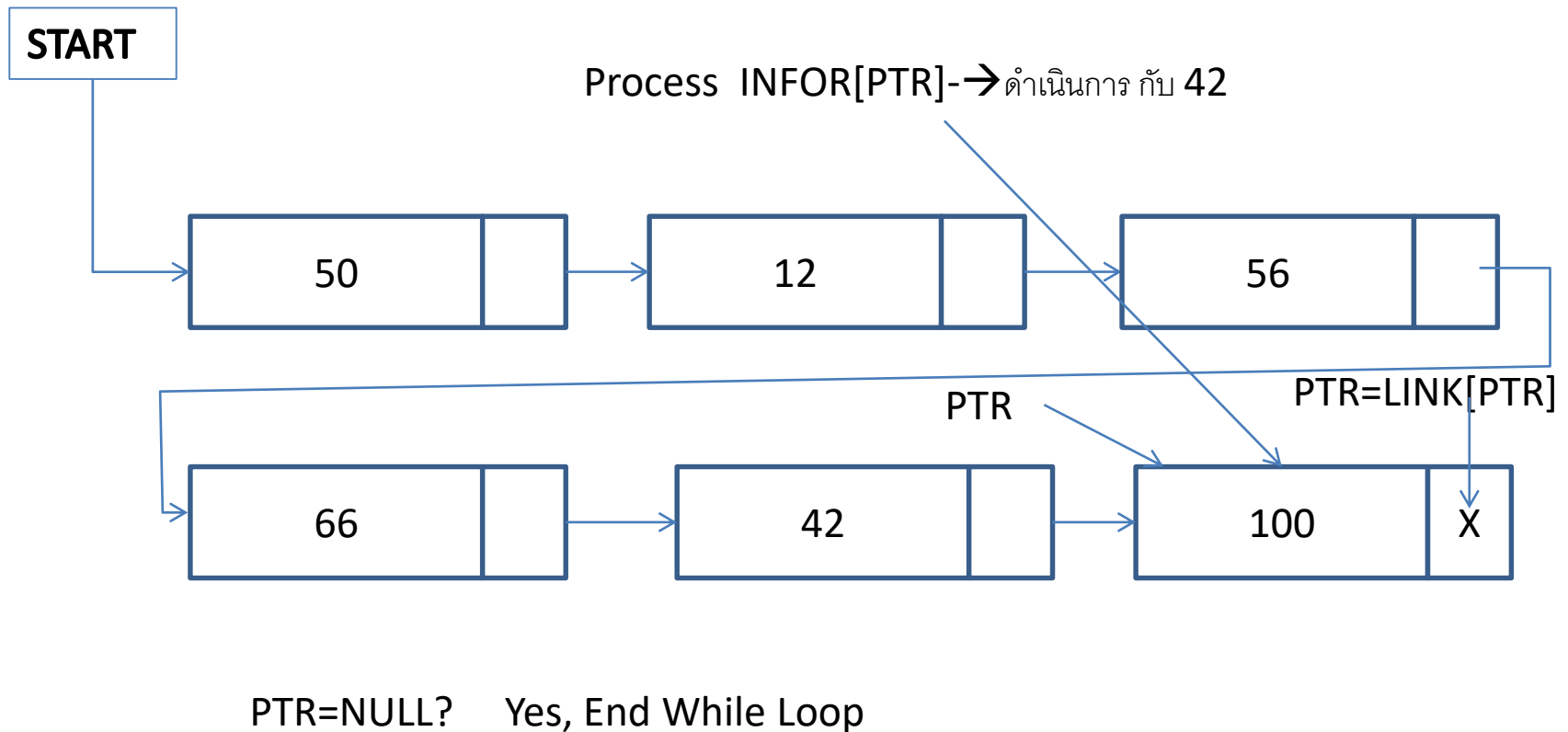
ตัวอย่างการท่องलि๊งค้ลิสต์ตามอัลกอริทึม



ตัวอย่างการท่องलि่ค้ลิสต์ตามอัลกอริทึม



ตัวอย่างการท่องलि่ค้ลิสต์ตามอัลกอริทึม



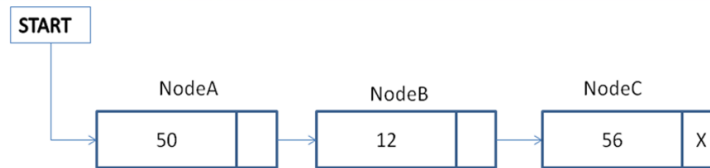
แนวทางการเขียนโปรแกรมสำหรับรายการโยงทางเดียว

การเขียนโปรแกรม สามารถอธิบายเป็นขั้นตอนเบื้องต้นได้ดังนี้

1. สร้างโครงสร้างโนด

```
1 public class Node {  
2     Object INFOR;  
3     Node LINK=null;  
4 }
```

2) สร้างส่วนโนดเก็บข้อมูล รายการโยง มีตัวชี้ PTR , START กำหนดให้ ตามตัวอย่าง

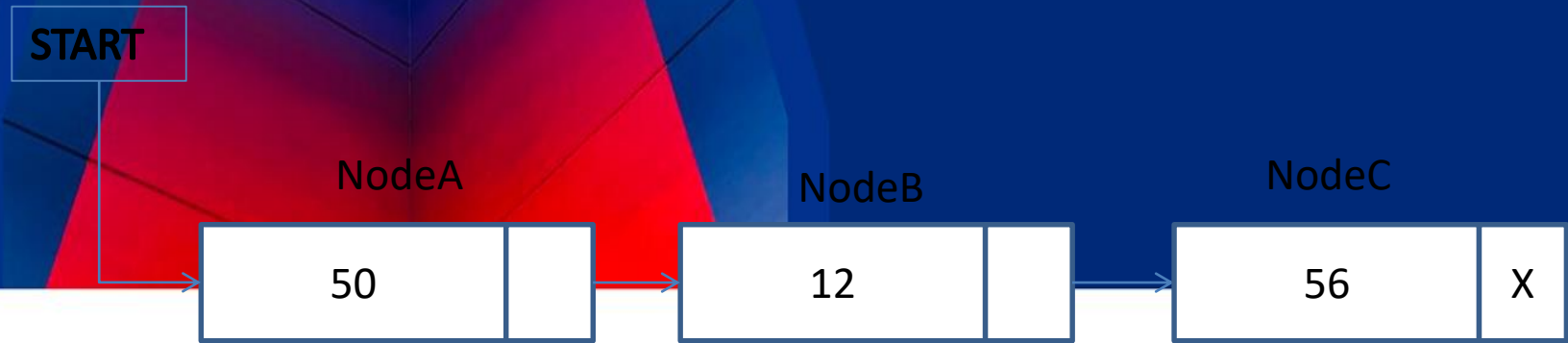


รูปที่ 6.16 แสดงรายการโยงที่ถูกสร้างโดยกำหนดป้ายชื่อ

```
1.      Node START=null;
2.      Node NodeA = new Node();
3.      NodeA.INFOR = 50;
4.      Node NodeB = new Node();
5.      NodeB.INFOR = 12;
6.      Node NodeC = new Node();
7.      NodeC.INFOR = 56;
8.      START.LINK = NodeA;
9.      NodeA.LINK = NodeB;
10.     NodeB.LINK=NodeC
11.     Node PTR=null;
12.     PTR=START;
```

3) เขียนส่วนการท่องตามอัลกอริทึม โดยแสดงดังตัวอย่างนี้

```
1.      while(PTR!=null){
2.          System.out.println(PTR.INFOR);
3.          PTR=PTR.LINK;
4.      }
```



- Node START=null;
- Node NodeA = new Node();
- NodeA.INFOR = 50;
- Node NodeB = new Node();
- NodeB.INFOR = 12;
- Node NodeC = new Node();
- NodeC.INFOR = 56;
- START.LINK = NodeA;
- NodeA.LINK = NodeB;
- NodeB.LINK=NodeC

- public class Algor51TraversingList {
- public static void main(String[] args) {
- Node START=null;
- Node NodeA = new Node();
- NodeA.INFOR = 50;
- Node NodeB = new Node();
- NodeB.INFOR = 12;
- Node NodeC = new Node();
- NodeC.INFOR = 56;
- START.LINK = NodeA;
- NodeA.LINK = NodeB;
- NodeB.LINK=NodeC
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- System.out.println(PTR.INFOR); //Apply PROCESS to INFOR[PTR]
- PTR=PTR.LINK;
- }
- }
- }

ส่วนการกำหนดค่าให้กับโนด

อัลกอริทึมหลัก

กิจกรรมนักศึกษา

- รันโปรแกรมทดสอบ
- ดัดแปลงให้เข้ากับโจทย์ที่นักศึกษากำหนดเอง

การค้นหาข้อมูลในรายการโยงทางเดียว

- การค้นหาข้อมูลในรายการโยงอาจแบ่งออกเป็นสองลักษณะ คือ การค้นหารายการโยงแบบไม่มีการจัดเรียงข้อมูลในรายการโยง และค้นหาจากรายการโยงที่มีการจัดเรียงแล้ว

การค้นหาในลิงค์ลิสต์ (Searching a Linked Lists)

- หลักการค้นหา คือ
 - กำหนดค่าเริ่มต้น จาก โหนดแรก
 - เปรียบข้อมูลว่า เท่ากับ ข้อมูลในโนดหรือไม่
 - หากเท่ากัน ให้รายงานตำแหน่งโนดนั้น แล้วออกจากการค้นหา
 - หากไม่เท่ากัน ให้เลื่อนพอยเตอร์ไปยังโนดถัดไป
 - ดำเนินการจนกระทั่งพอยเตอร์ที่ชี้ต่อไปนั้น เป็นค่า **null** (ตัวสุดท้าย)

การค้นหารายการโยงที่ยังไม่ได้จัดเรียง

- การค้นหาข้อมูลในรายการโยงกรณีที่ยังไม่มีการจัดเรียงข้อมูล เริ่มจากกำหนดค่าเริ่มต้นที่จะค้นหาเปรียบเทียบกับข้อมูลที่ต้องการ นำไปเทียบกับโนดแรก โดยเทียบข้อมูลว่า เท่ากับข้อมูลในโนดหรือไม่ หากเท่ากัน ให้รายงานตำแหน่งโนดนั้น แล้วออกจากการค้นหา แต่หากไม่เท่ากัน ให้เลื่อนตัวชี้ ไปยังโนดถัดไป ดำเนินการเทียบกับข้อมูลในตำแหน่งตัวชี้ที่อยู่จนกระทั่งพบ หรือตัวชี้ที่ชี้ต่อไปนั้นเป็นค่า **NULL** ซึ่งเป็นข้อมูลตัวสุดท้าย และหากเป็นกรณีนี้แสดงว่า ข้อมูลที่ต้องการค้นหานั้นไม่มีในรายการโยงที่ทำการค้นอยู่ เพื่อความเข้าใจสำหรับการใช้งานตัวแปรและอธิบายอัลกอริทึม ได้กำหนดตัวแปรเพิ่มขึ้นในนิยามที่ **6.3**

แนวทางของอัลกอริทึม

- กำหนด **ITEM** คือข้อมูลที่ต้องการค้นหา
- **LOC** คือตำแหน่งที่พบ กำหนดค่าเริ่มต้นเป็น **NULL**
- กำหนด **PTR=START**
- วนดำเนินการ ในขณะที่ **PTR!=NULL**
 - เทียบ **ITEM = INFOR[PTR]** หรือไม่
 - หากเท่ากัน **LOC=PTR**
 - หากไม่เท่ากัน **PTR=LINK[PTR]**
 - วนกลับไปดำเนินการ

นิยามที่ 6.3 กำหนดให้ LOC คือ ตัวชี้สำหรับชี้ไปยังข้อมูล ITEM ใดๆ ที่ได้จากการค้นหา มีค่าเริ่มต้นคือ NULL

แสดงแนวคิดการค้นหาและอัลกอริทึม ดังนี้

ค้นหาข้อมูล ITEM ในรายการโยง ที่ยังไม่มีการจัดเรียงข้อมูล โดยจะต้องมี LIST, PTR, START, LOC, ITEM เป็นข้อมูลเข้า LOC เป็นข้อมูลออก มีขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. กำหนด ITEM คือข้อมูลที่ต้องการค้นหา
2. LOC คือตำแหน่งที่พบ กำหนดค่าเริ่มต้นเป็น NULL
3. กำหนด PTR=START
4. วงดำเนินการ ในขณะที่ PTR!=NULL
5. เทียบ ITEM = INFOR[PTR] หรือไม่
 - 5.1 หากเท่ากัน LOC=PTR ค้นค่า LOC
 - 5.2 หากไม่เท่ากัน PTR=LINK[PTR]
วนกลับไปขั้นตอน 4
6. ค้นค่า LOC

Algorithm 6.2 SearchUnSortedList

Input : LIST, PTR, START, LOC, ITEM

Output : LOC

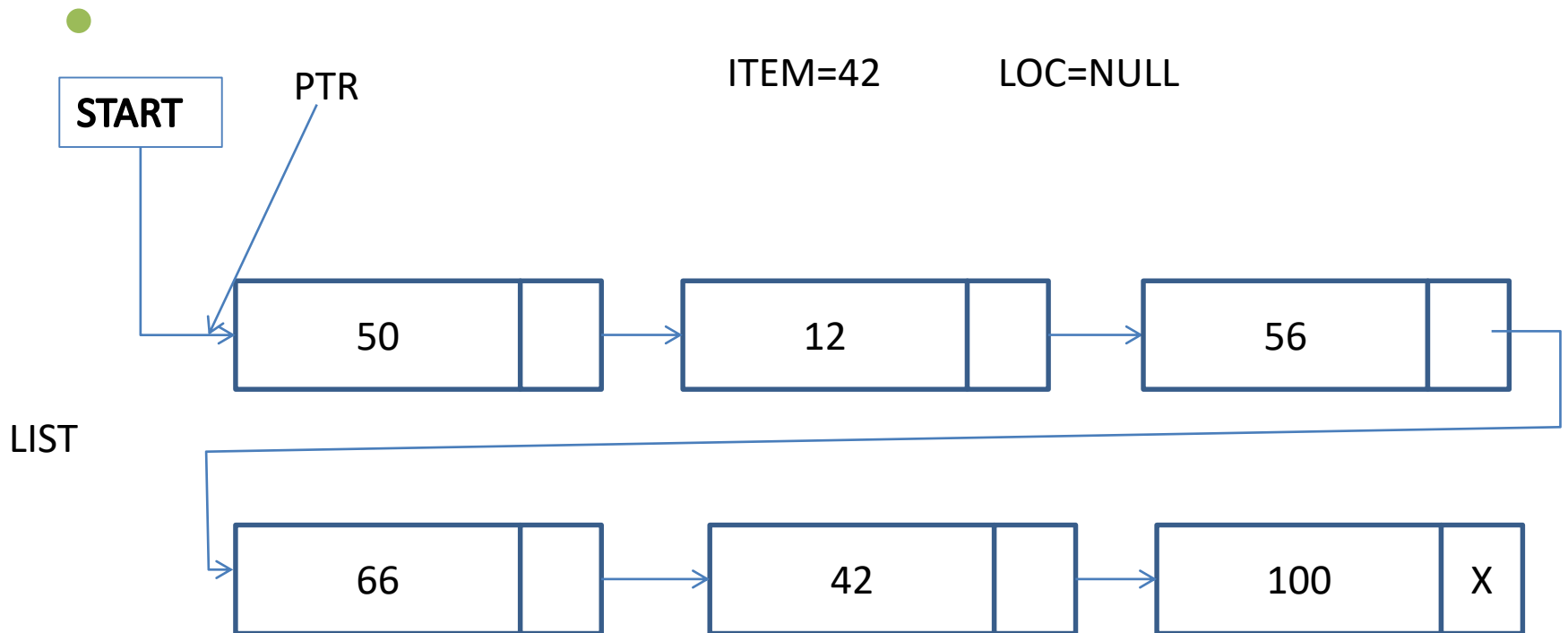
1. PTR:=START
2. While PTR!=NULL Do
3. IF ITEM= INFOR[PTR] Then
4. LOC := PTR
5. Return LOC
6. Else
7. PTR:=LINK[PTR]
8. End IF
9. LOC:=NULL
10. End While
11. Return LOC



ตัวอย่าง 6.4 กำหนดให้ข้อมูลในรายการโยง คือ 50, 12, 56, 66, 42, 100 จงแสดงตัวอย่าง
แสดงการค้นหา ITEM=42 ตามแนวทางของอัลกอริทึมด้วยรูปภาพ

เฉลย: เบื้องต้นต้องกำหนดให้มีการสร้างรายการโยงให้เรียบร้อยก่อน แล้วจึงแสดงการ
ค้นหาดังรูปที่ 6.16

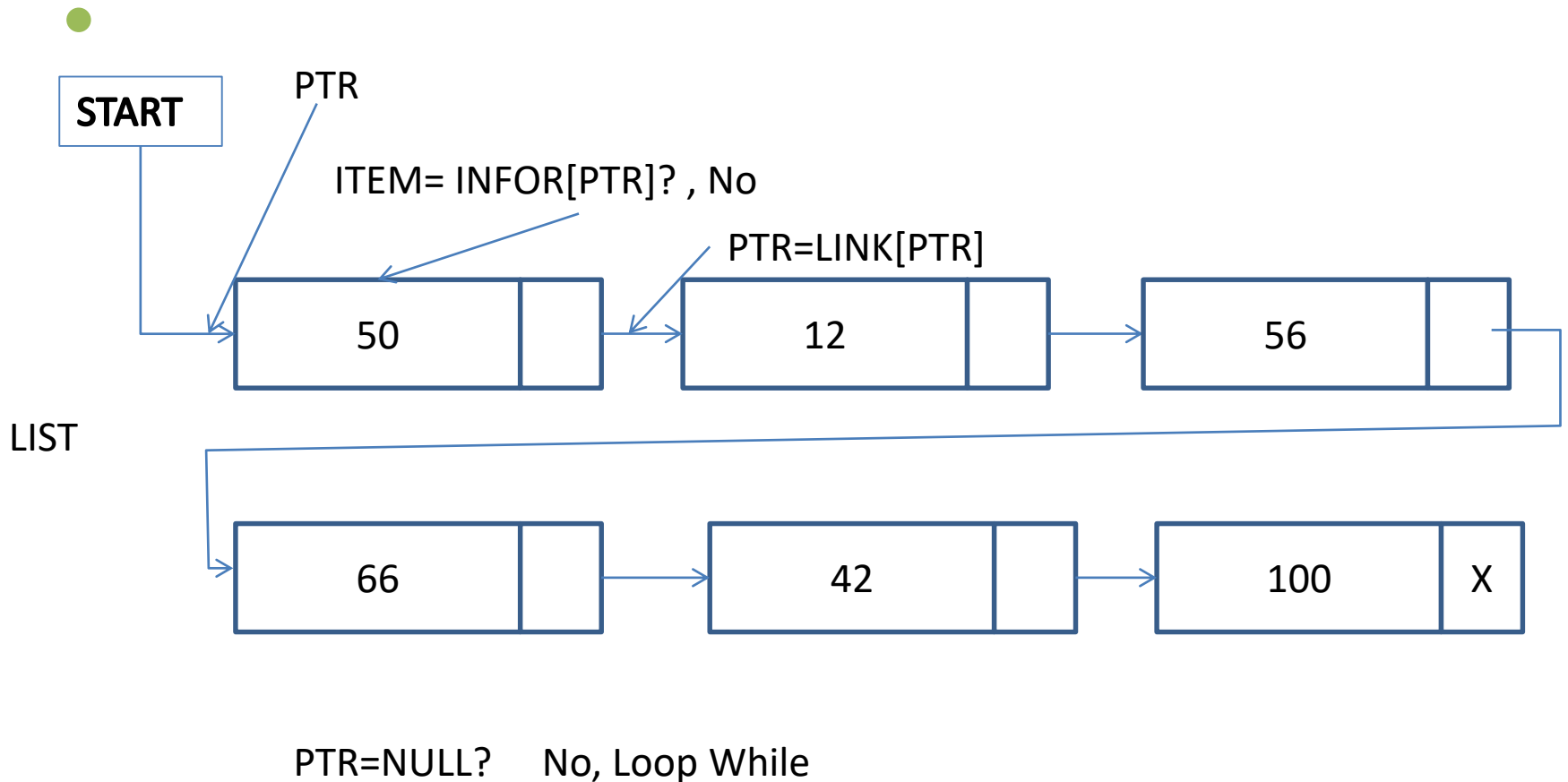
แนวทางการค้นหาอัลกอริทึม



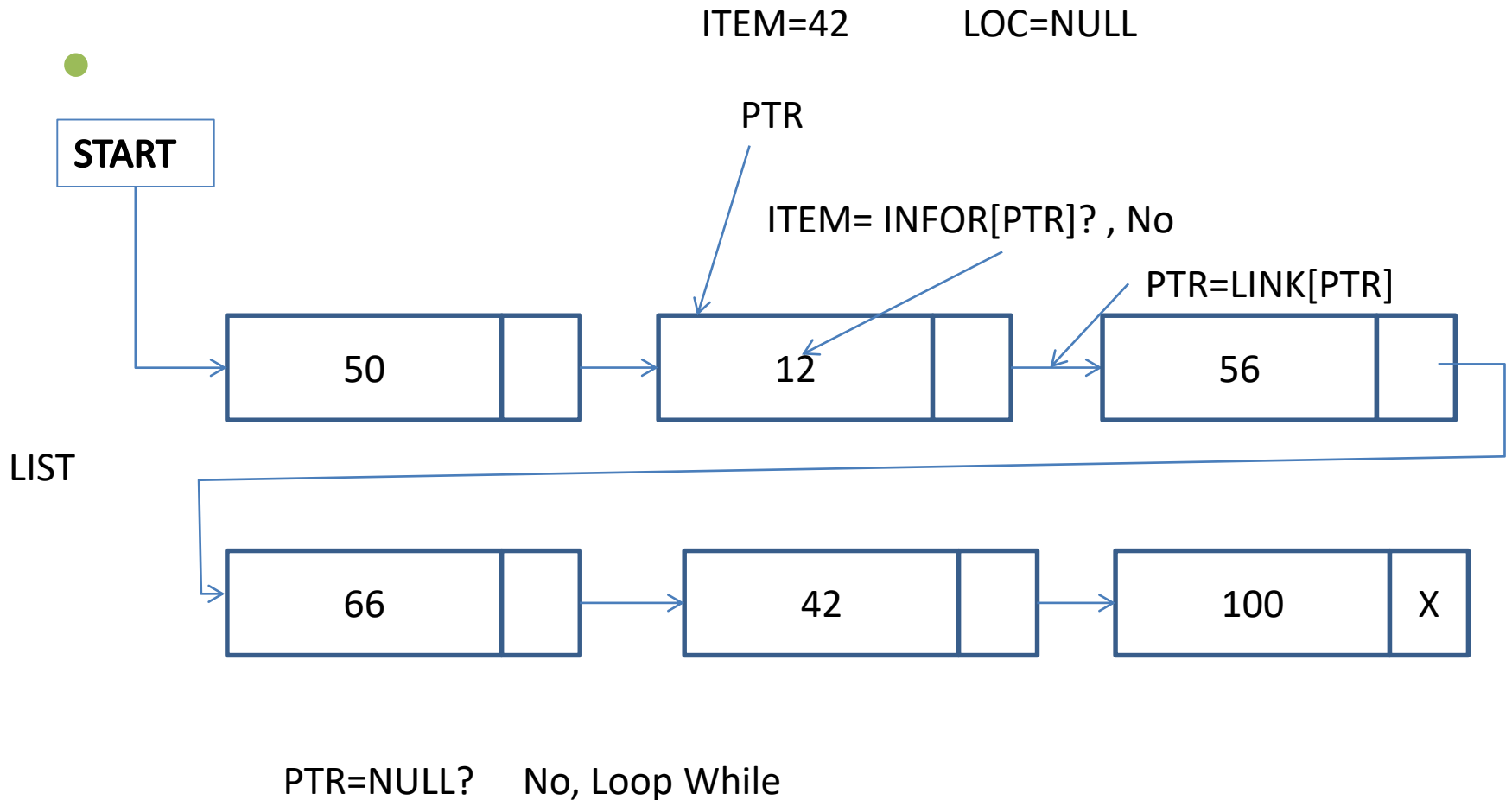
แนวทางการค้นหาอัลกอริทึม

ITEM=42

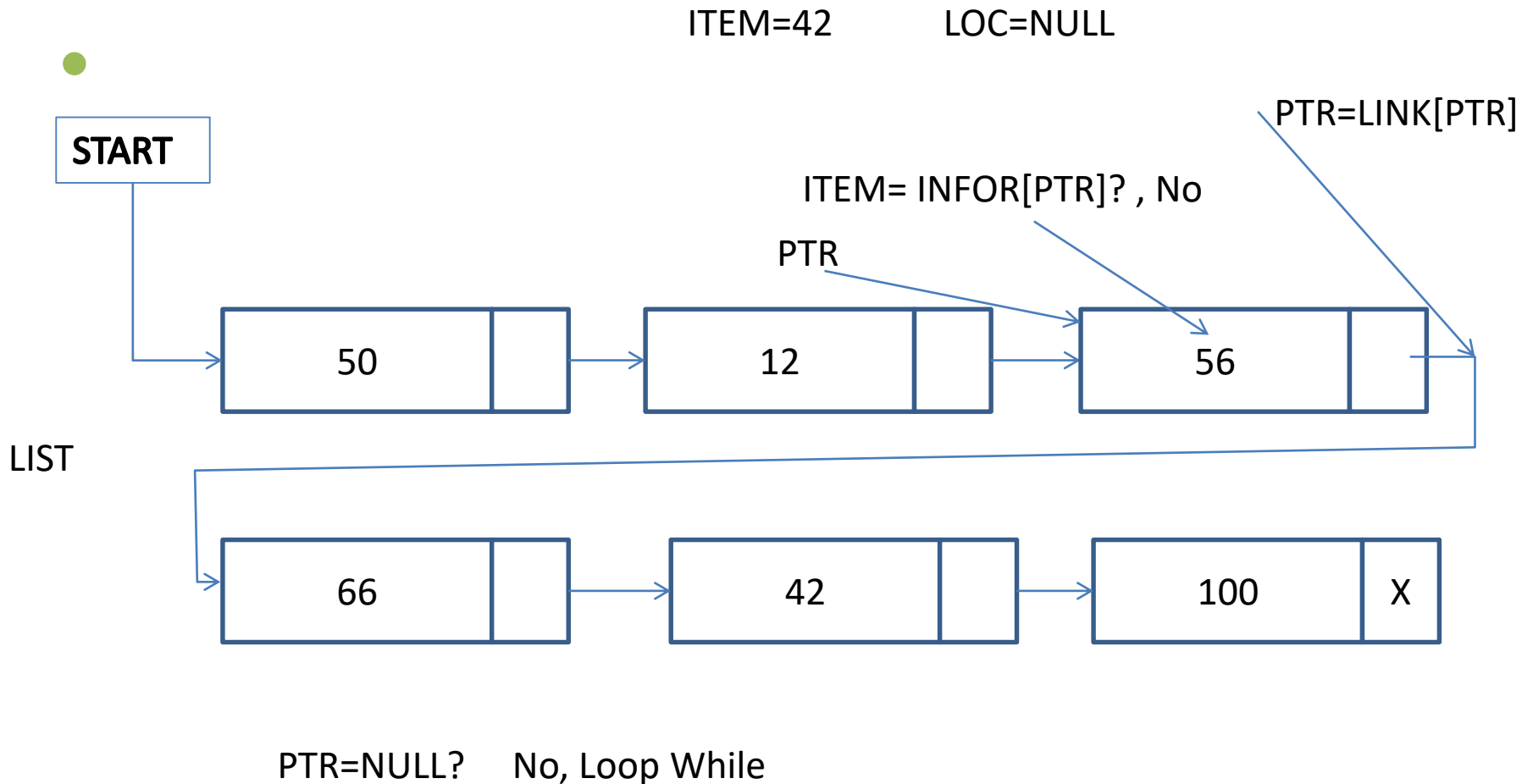
LOC=NULL



แนวทางการค้นหาตามอัลกอริทึม



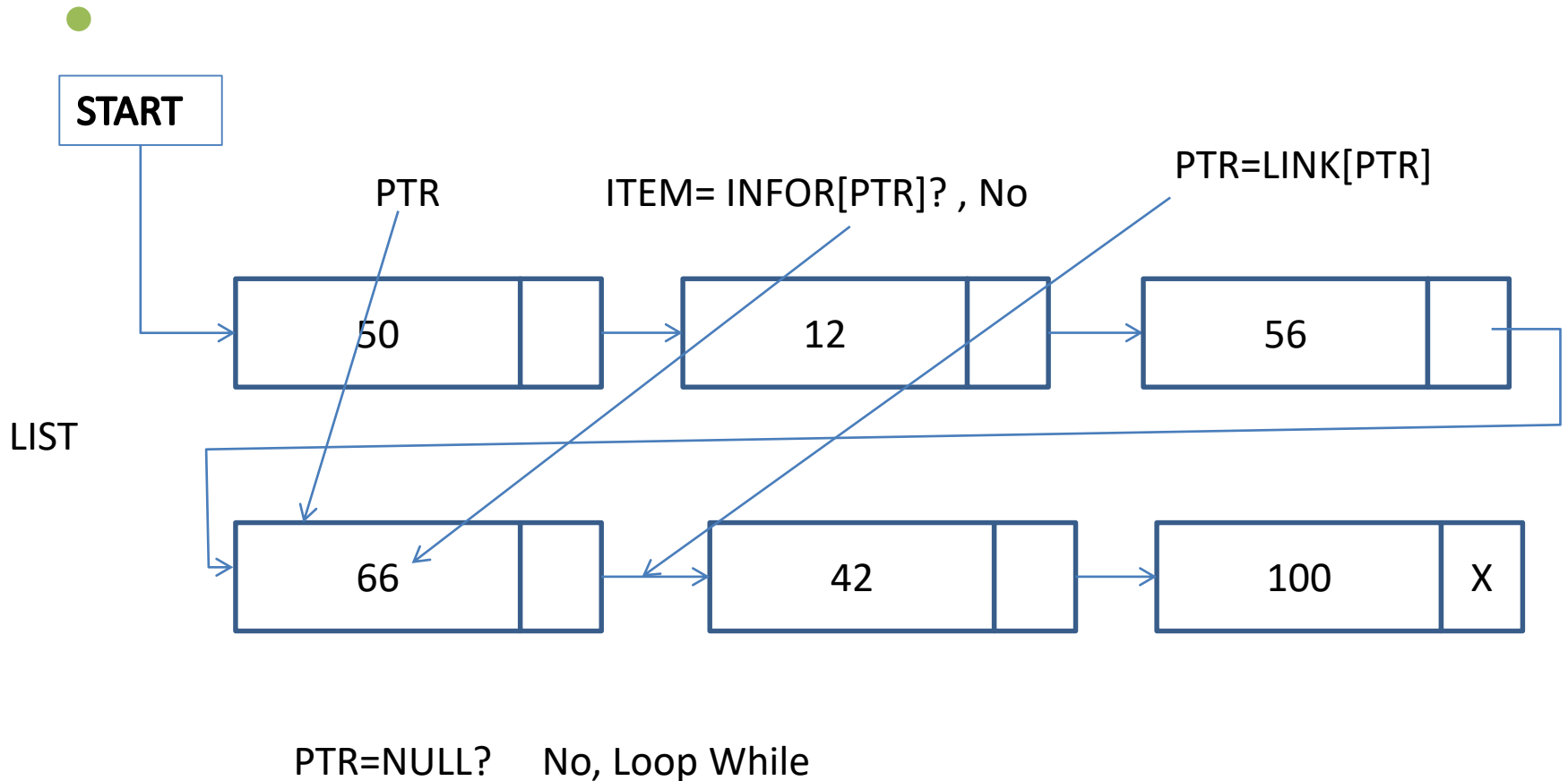
แนวทางการค้นหาตามอัลกอริทึม



แนวทางการค้นหาอัลกอริทึม

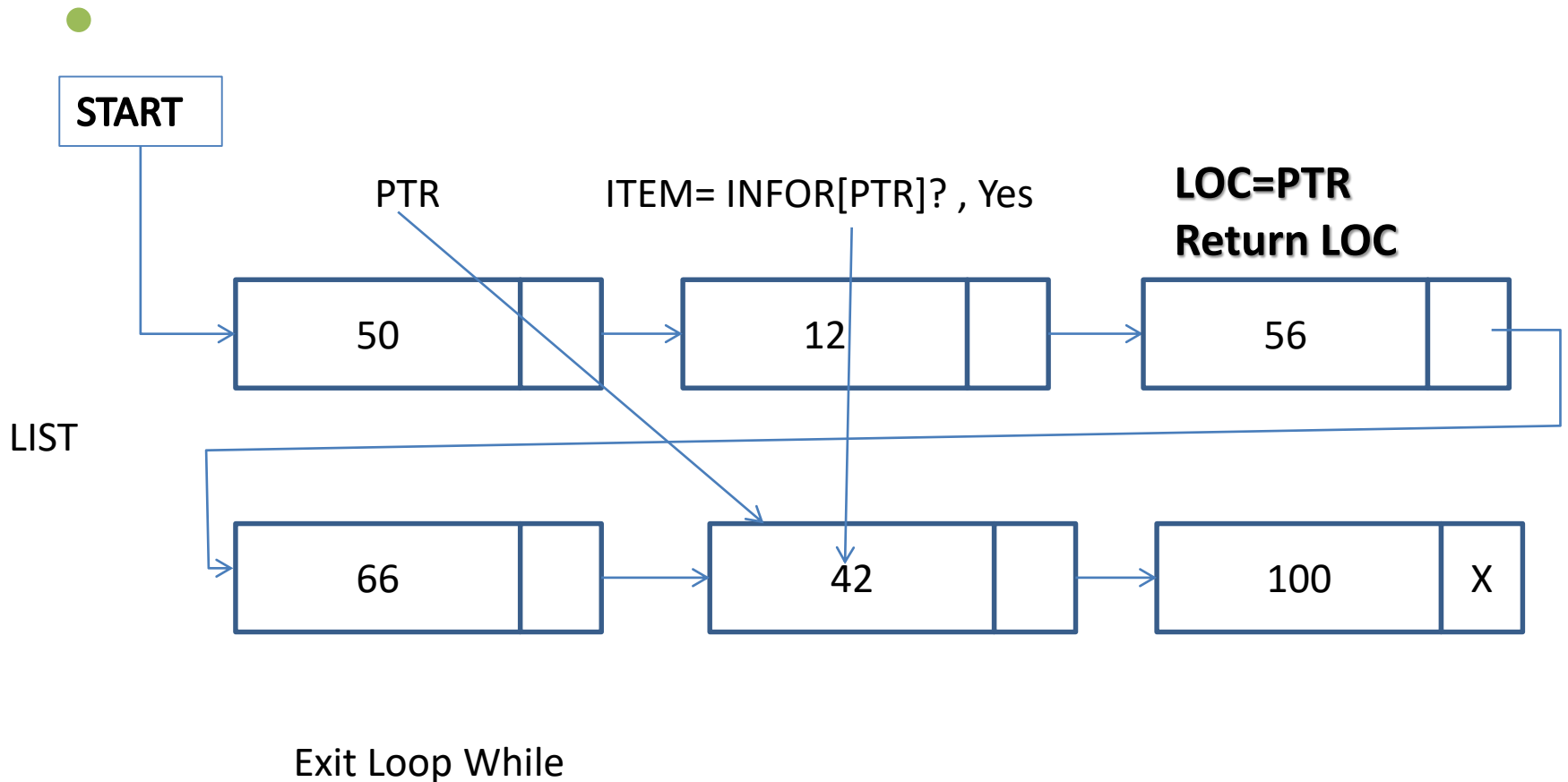
ITEM=42

LOC=NULL



แนวทางการค้นหาอัลกอริทึม

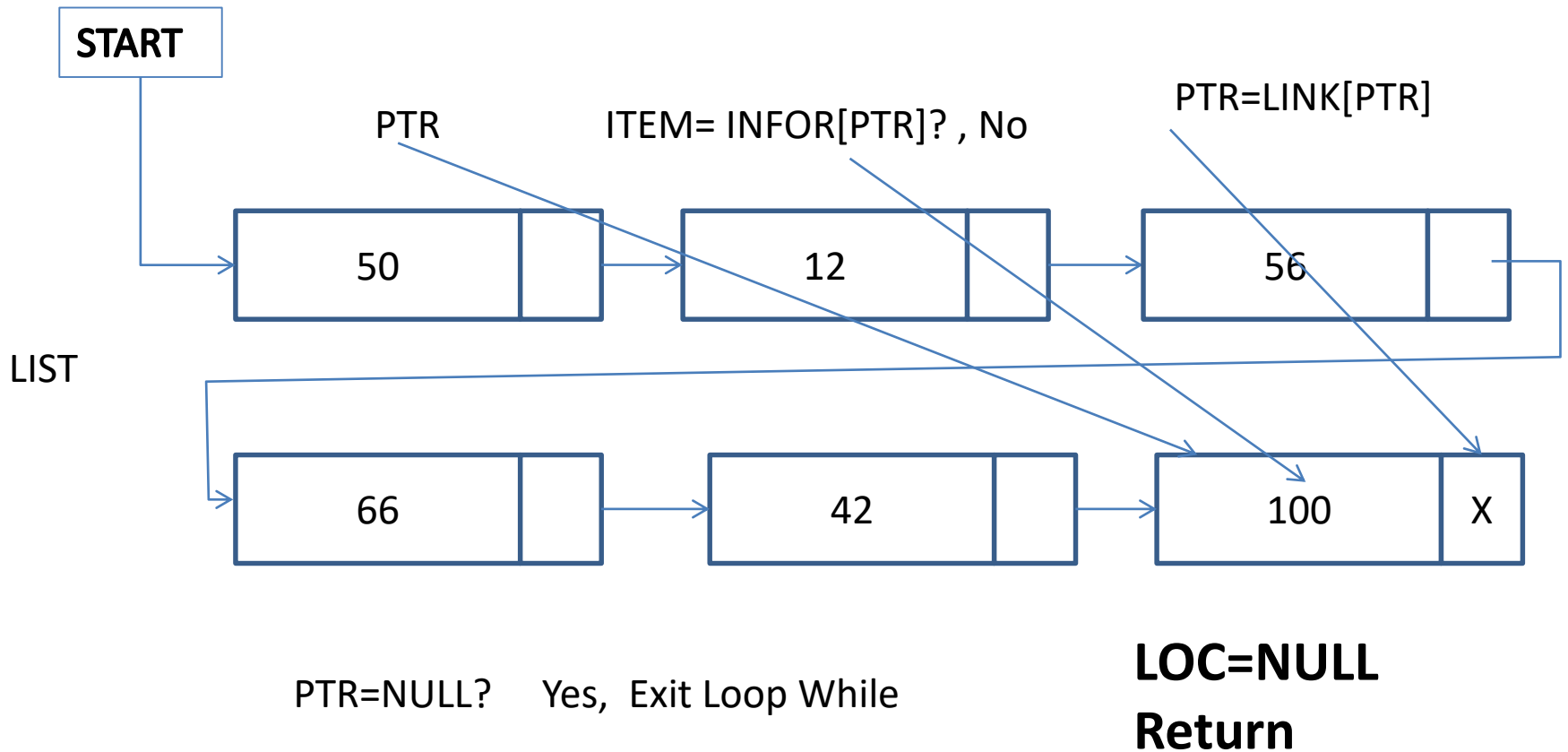
ITEM=42



แนวทางการค้นหาตามอัลกอริทึม

• กรณีไม่พบ

ITEM=90





อิมพลีเมนต์

```

package linklistimplementation;

public class Algor52SEARCH {
    public static void main(String[] args) {
        Node START = new Node(); START.INFOR = 10;
        Node NodeA = new Node(); NodeA.INFOR = 20;
        Node NodeB = new Node(); NodeB.INFOR = 30;
        Node NodeC = new Node(); NodeC.INFOR = 40;
        Node NodeD = new Node(); NodeD.INFOR = 50;
        Node NodeE = new Node(); NodeE.INFOR = 60;
        Node NodeF = new Node(); NodeF.INFOR = 70;
        Node NodeG = new Node(); NodeG.INFOR = 80;
        Node NodeH = new Node(); NodeH.INFOR = 90;
        Node NodeI = new Node(); NodeI.INFOR = 100;
        START.LINK = NodeA; NodeA.LINK = NodeB; NodeB.LINK = NodeC;
        NodeC.LINK = NodeD; NodeD.LINK = NodeE; NodeE.LINK = NodeF;
        NodeF.LINK = NodeG; NodeG.LINK = NodeH; NodeH.LINK = NodeI;

        //Algorrithm 5.2 SEARCH
        int ITEM = 140;
        Node LOC=null;
        Node PTR=null;
        PTR=START;
        while(PTR!=null){
            if(ITEM==(int)PTR.INFOR) {
                LOC = PTR;
                break;
            }else
                PTR=PTR.LINK;
        }
        //Show Result;
        if(LOC ==null)
            System.out.println("Not found.");
        else
            System.out.println("Found.");
    }
}

```

- public class Algor52SEARCH {
- public static void main(String[] args) {
- Node START = new Node(); START.INFOR = 10;
- Node NodeA = new Node(); NodeA.INFOR = 20;
- ...
- Node NodeI = new Node(); NodeI.INFOR = 100;
- START.LINK = NodeA; NodeA.LINK = NodeB; NodeB.LINK = NodeC;
- NodeC.LINK = NodeD; NodeD.LINK = NodeE; NodeE.LINK = NodeF;
- NodeF.LINK = NodeG; NodeG.LINK = NodeH; NodeH.LINK = NodeI;

ส่วนการกำหนดค่าให้กับโนด

- //Algorrithm 5.2 SEARCH
- int ITEM = 140;
- Node LOC=null;
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- if(ITEM==(int)PTR.INFOR) {
- LOC = PTR;
- break;
- }else
- PTR=PTR.LINK;
- }
- //Show Result;
- if(LOC ==null)
- System.out.println("Not found.");
- else
- System.out.println("Found.");
- }
- }

อัลกอริทึมหลัก

กิจกรรมนักศึกษา

- รันโปรแกรมทดสอบ
- เขียนโปรแกรมเพิ่มเติมตามโจทย์ที่กำหนด

การค้นหาถึงผลลัพธ์ที่มีการจัดเรียง

- การค้นหาใช้หลักการท่องไป แล้วเปรียบเทียบเช่นเดียวกันกับแบบไม่มีการจัดเรียง
- เมื่อพบก็รายงานตำแหน่งที่พบ แล้วจบการค้นหา
- แต่เมื่อยังไม่พบจะต้องค้นไป จนกระทั่ง พบตัวที่มากกว่า หรือน้อยกว่า แล้วแต่ว่า การจัดเรียงนั้นเรียงข้อมูลแบบใด
- และหากยังไม่พบให้ค้นไปจนกระทั่งตัวสุดท้าย แล้วรายงานว่าไม่พบข้อมูล

แนวทางอัลกอริทึม

- กำหนด $PTR = START$
- วนทำงานในขณะที่ $PTR \neq NULL$
 - ถ้า $ITEM < INFO[PTR]$ เลื่อนตัวชี้ต่อไป $PTR = LINK[PTR]$
 - ถ้า $ITEM = INFO[PTR]$ กำหนด $LOC = PTR$ และจบการค้น
 - แต่ถ้ายังไม่ใช่ ให้กำหนด $LOC = NULL$ และจบการค้นเพราะว่า ตัวที่ค้นเกินค่าที่กำหนดไว้แล้ว (น้อยกว่าหรือมากกว่าค่าที่ต้องการค้นแล้ว)
- หากออกจากลูป แล้วยังไม่เข้ากรณีใดๆ ที่ผ่านมา ให้กำหนด $LOC = NULL$ แสดงว่าการค้นหาค่า $ITEM$ นั้นยังน้อยกว่าหรือมากกว่าค่าใน $LIST$

ค้นหาข้อมูล ITEM ในรายการโยง ที่จัดเรียงข้อมูลไว้แล้ว โดยจะต้องมี LIST, PTR, START, LOC, ITEM เป็นข้อมูลเข้า LOC เป็นข้อมูลออก ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. กำหนด PTR=START
2. วนทำงานในขณะที่ PTR!= NULL
 - 2.1 ถ้า ITEM<INFOR[PTR] เลื่อนตัวชี้ต่อไป PTR=LINK[PTR]
 - 2.2 ถ้า ITEM=INFOR[PTR] กำหนด LOC=PTR และจบการค้น
 - 2.3 แต่ถ้ายังไม่ใช่ ให้กำหนด LOC=NULL และจบการค้น เพราะว่า ตัวที่ค้นเกินค่าที่กำหนดไว้แล้ว (น้อยกว่าหรือมากกว่าค่าที่ต้องการค้นแล้ว)
3. หากออกจากลูป แล้วยังไม่เข้ากรณีใดๆ ที่ผ่านมาให้กำหนด LOC=NULL แสดงว่าการค้นหาค่า ITEM นั้นยังน้อยกว่าหรือมากกว่าค่าใน LIST

Algorithm 6.3 : SEARCHSortedLists

Input : LIST, PTR, START, LOC, ITEM

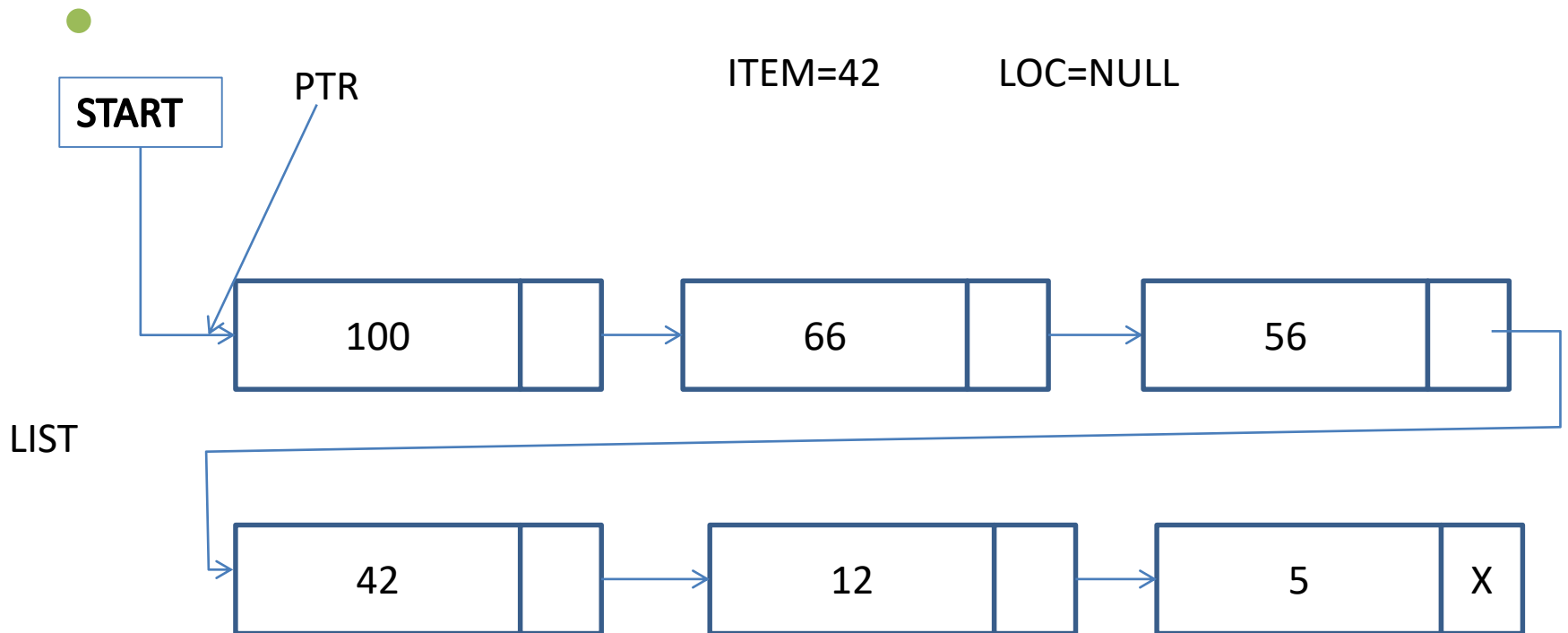
Output : LOC

1. PTR:=START
2. While PTR!=NULL Do
3. IF ITEM < INFOR[PTR] Then
4. PTR:=LINK[PTR]
5. Else
6. IF ITEM=INFOR[PTR] Then
7. LOC := PTR
8. Return LOC
9. Else
10. LOC:=NULL
11. Return LOC
12. End IF
13. LOC:=NULL
14. End While
15. Return LOC

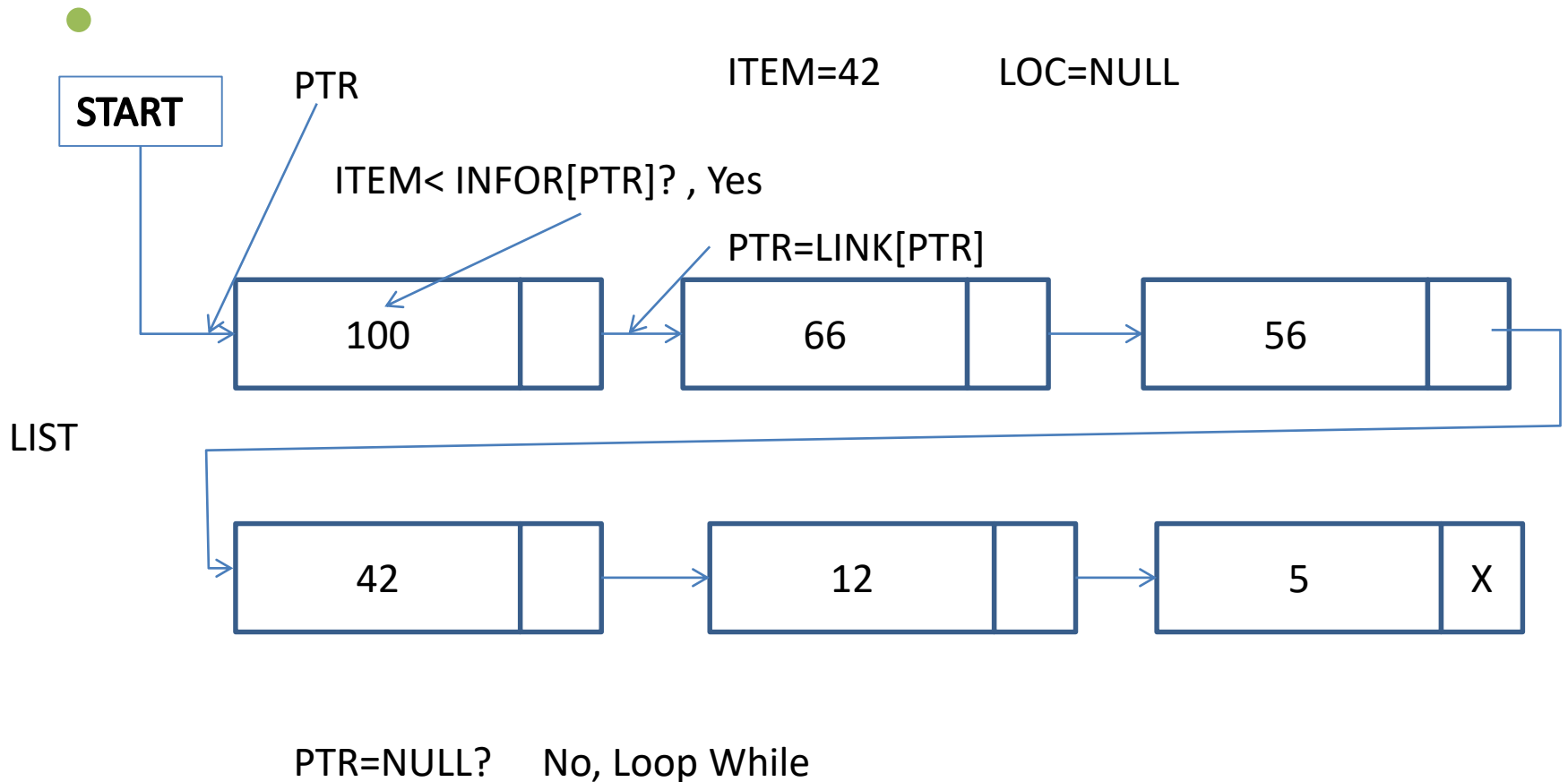


ตัวอย่าง 6.6 แสดงการค้นหาข้อมูลรายการโยงที่มีการจัดเรียงอยู่แล้ว คือ 100, 66, 56, 42, 12, 5 ตามลำดับ

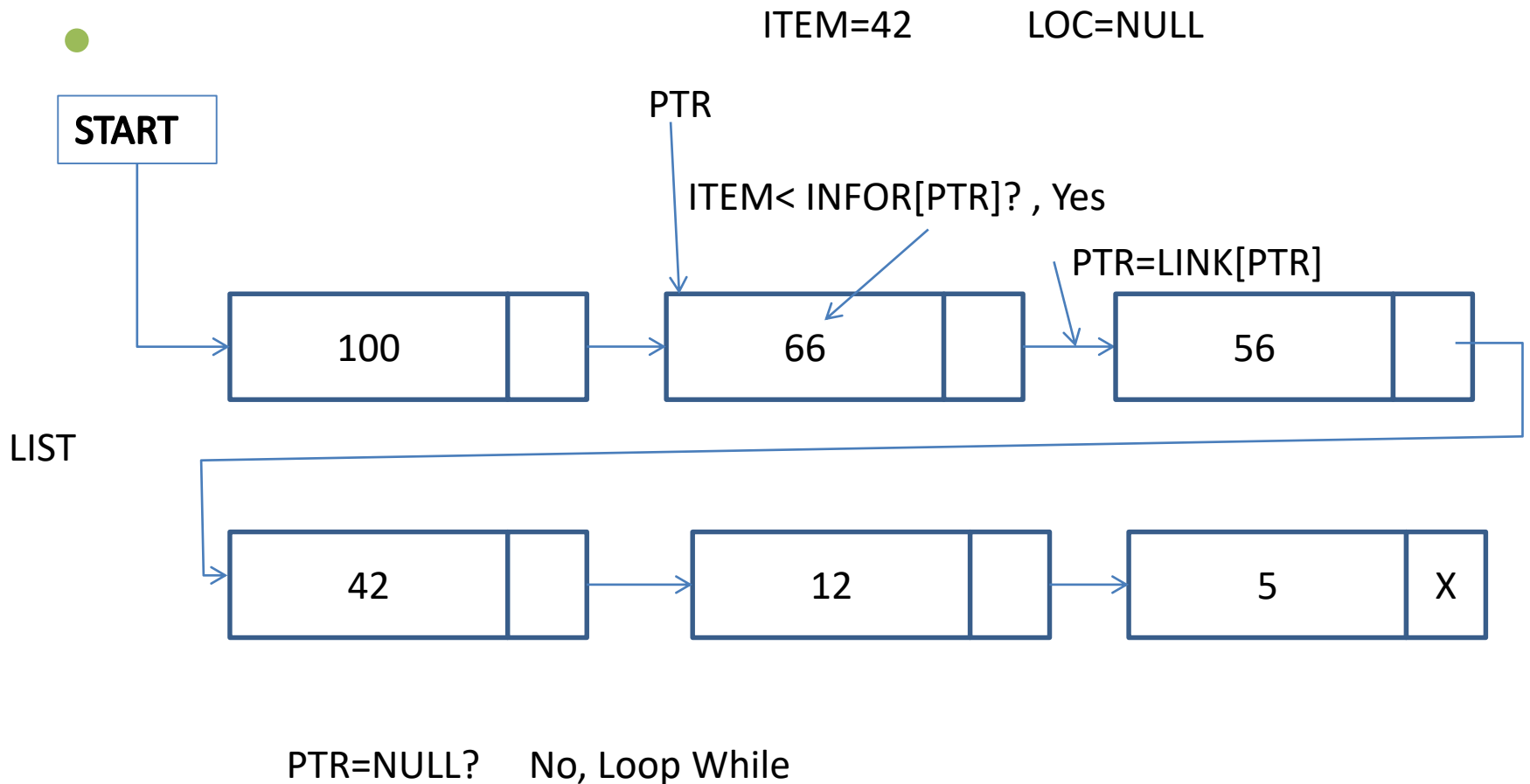
แนวทางการค้นตามอัลกอริทึม



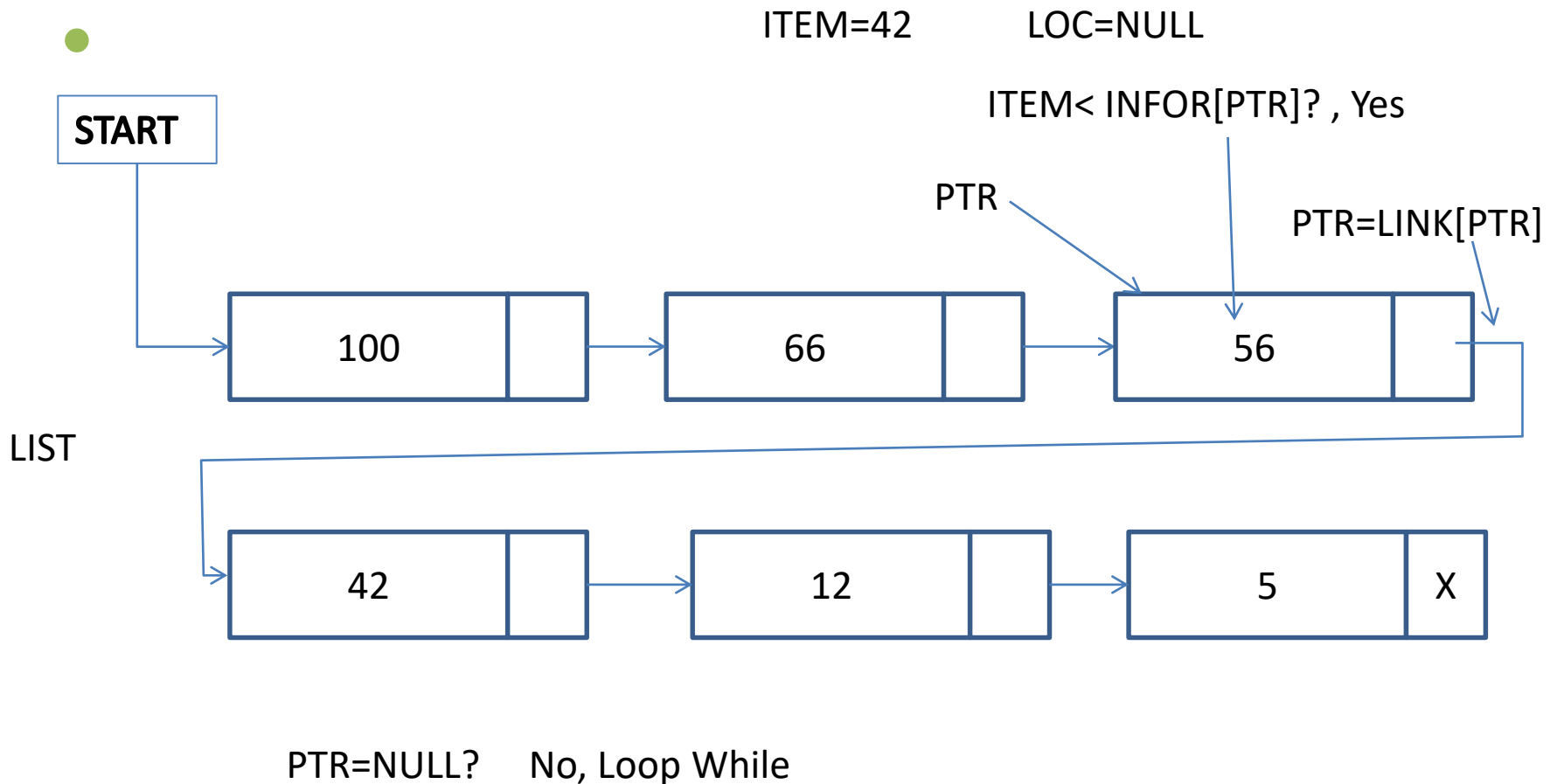
แนวทางการค้นหาอัลกอริทึม



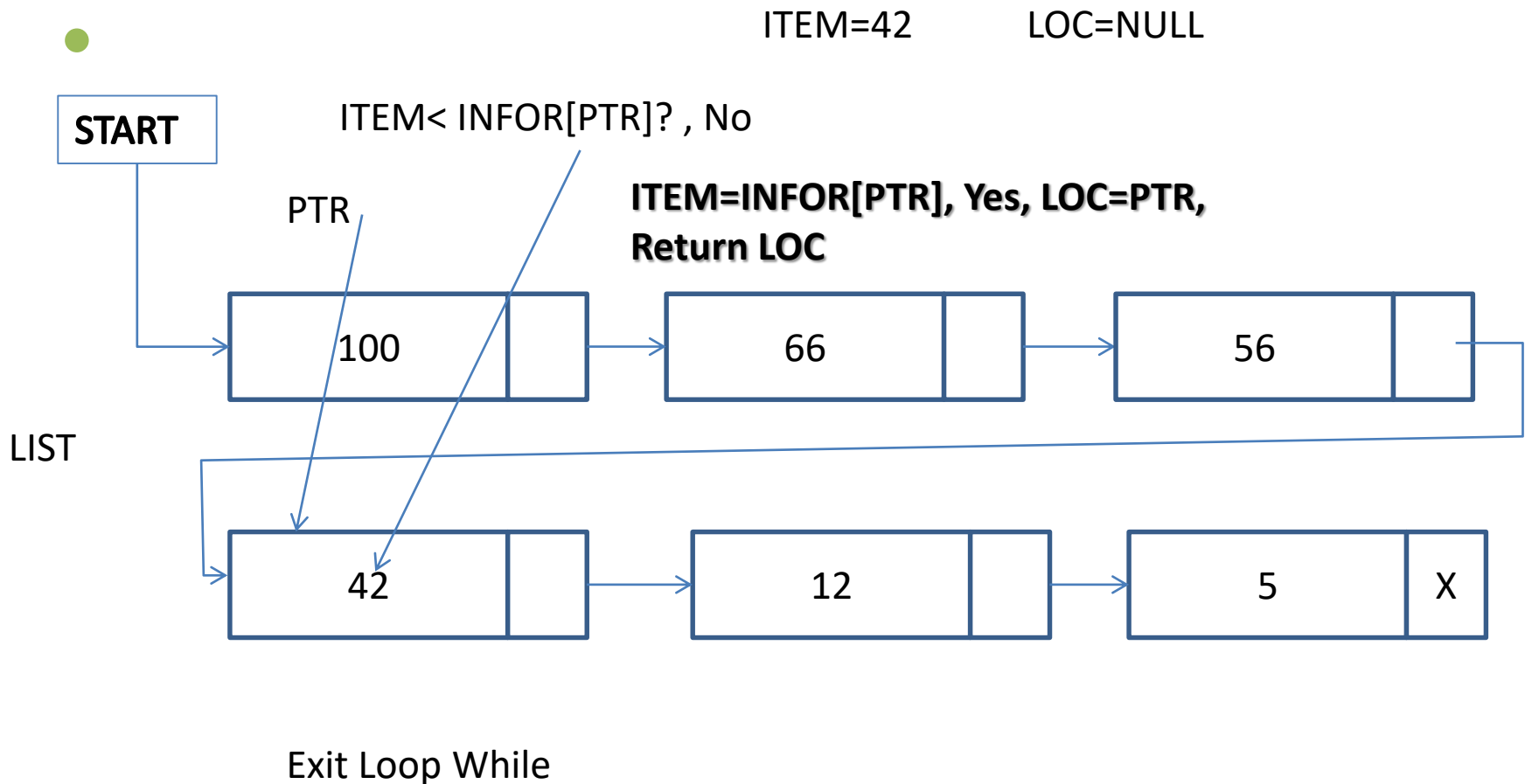
แนวทางการค้นตามอัลกอริทึม



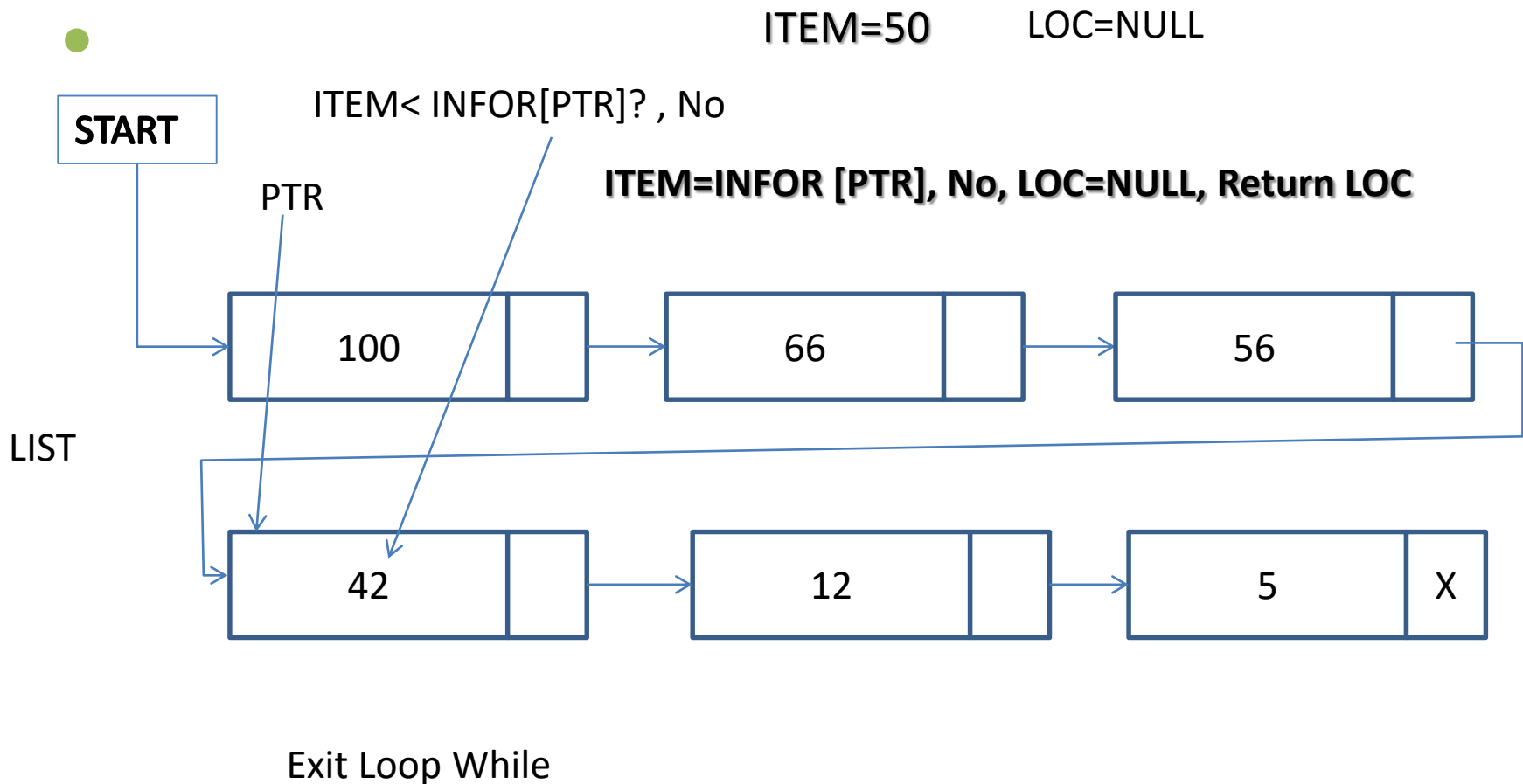
แนวทางการค้นหาอัลกอริทึม



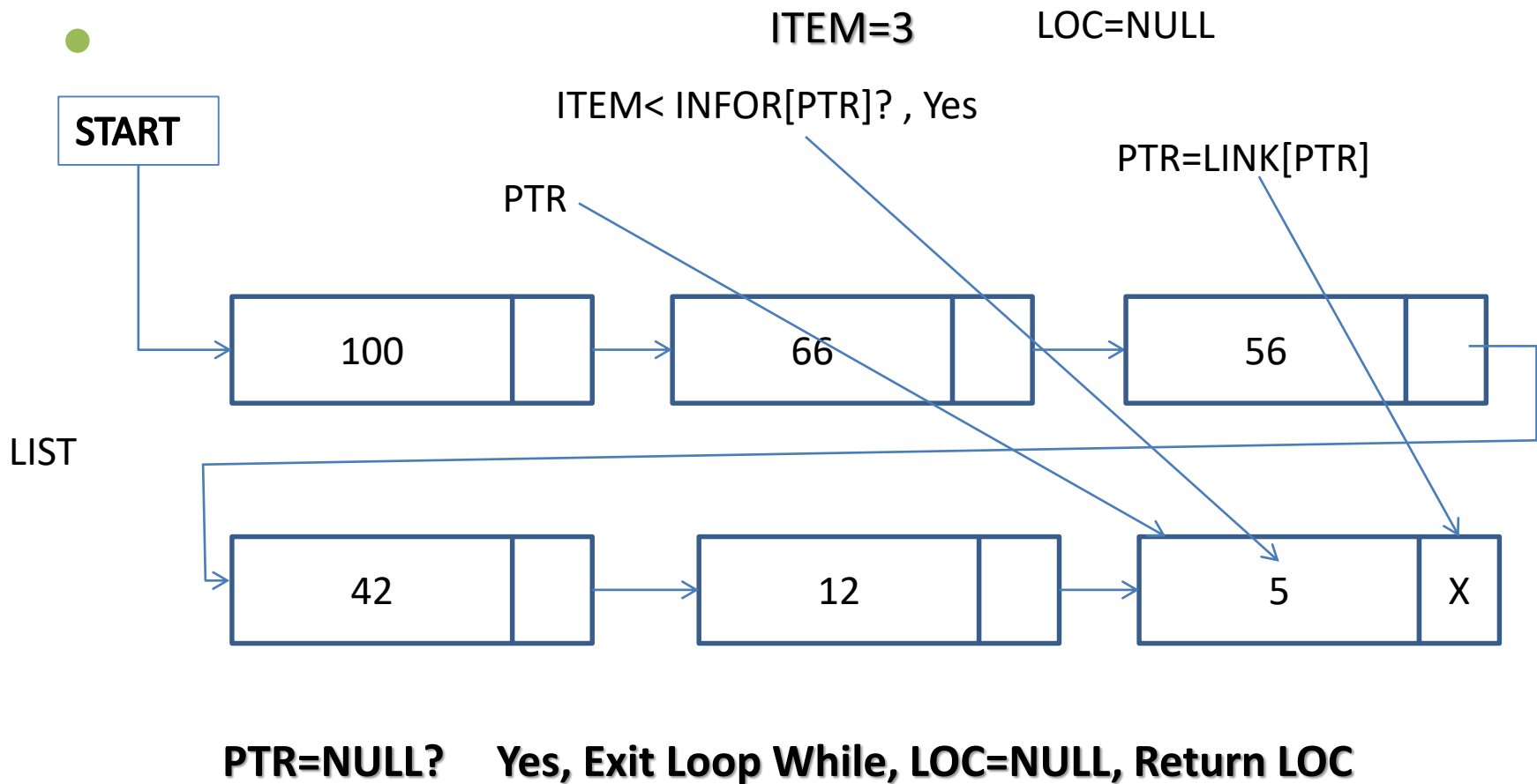
แนวทางการค้นหาอัลกอริทึม (พบ)



แนวทางการค้นหาอัลกอริทึม (ไม่พบ)



แนวทางการค้นหาอัลกอริทึม(ไม่พบ)





อิมพลีเมนต์

```

• public class Algor53SRCHSL {
•
•     public static void main(String[] args) {
•         SEARCH(40);
•     }//end main

•
•     //Method of Algorithm 5.2
•     public static void SEARCH(int item) {
•         Node START = new Node(); START.INFOR = 100;
•         Node NodeA = new Node(); NodeA.INFOR = 90;
•         Node NodeB = new Node(); NodeB.INFOR = 80;
•         Node NodeC = new Node(); NodeC.INFOR = 70;
•         Node NodeD = new Node(); NodeD.INFOR = 60;
•         Node NodeE = new Node(); NodeE.INFOR = 50;
•         Node NodeF = new Node(); NodeF.INFOR = 40;
•         Node NodeG = new Node(); NodeG.INFOR = 30;
•         Node NodeH = new Node(); NodeH.INFOR = 20;
•         Node NodeI = new Node(); NodeI.INFOR = 10;

•
•         START.LINK = NodeA; NodeA.LINK = NodeB; NodeB.LINK = NodeC;
•         NodeC.LINK = NodeD; NodeD.LINK = NodeE; NodeE.LINK = NodeF;
•         NodeF.LINK = NodeG; NodeG.LINK = NodeH; NodeH.LINK = NodeI;

•
•         //Algorrithm 5.2 SEARCH
•         int ITEM = item;
•         Node LOC=null;
•         Node PTR=null;
•         PTR=START;
•         while(PTR!=null){
•             if(ITEM<(int)PTR.INFOR) {
•                 PTR=PTR.LINK;
•             }else
•                 if(ITEM==(int)PTR.INFOR){
•                     LOC = PTR;
•                     break;
•                 }else{
•                     LOC=null;
•                     break;
•                 }
•         }
•         }//end while
•         //Show Result;
•         if(LOC ==null)

```

- `public static void SEARCH(int item) {`
- `Node START = new Node(); START.INFOR = 100;`
- `Node NodeA = new Node(); NodeA.INFOR = 90;`
- `Node NodeB = new Node(); NodeB.INFOR = 80;`
- `Node NodeC = new Node(); NodeC.INFOR = 70;`
- `Node NodeD = new Node(); NodeD.INFOR = 60;`
- `Node NodeE = new Node(); NodeE.INFOR = 50;`
- `Node NodeF = new Node(); NodeF.INFOR = 40;`
- `Node NodeG = new Node(); NodeG.INFOR = 30;`
- `Node NodeH = new Node(); NodeH.INFOR = 20;`
- `Node NodeI = new Node(); NodeI.INFOR = 10;`
- `START.LINK = NodeA; NodeA.LINK = NodeB; NodeB.LINK = NodeC;`
- `NodeC.LINK = NodeD; NodeD.LINK = NodeE; NodeE.LINK = NodeF;`
- `NodeF.LINK = NodeG; NodeG.LINK = NodeH; NodeH.LINK = NodeI;`

- int ITEM = item;
- Node LOC=null;
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- if(ITEM<(int)PTR.INFOR) {
- PTR=PTR.LINK;
- }else
- if(ITEM==(int)PTR.INFOR){
- LOC = PTR;
- break;
- }else{
- LOC=null;
- break;
- }
- }//end while
- //Show Result;
- if(LOC ==null)
- System.out.println("Not found.");
- else
- System.out.println("Found.");
- }
- }

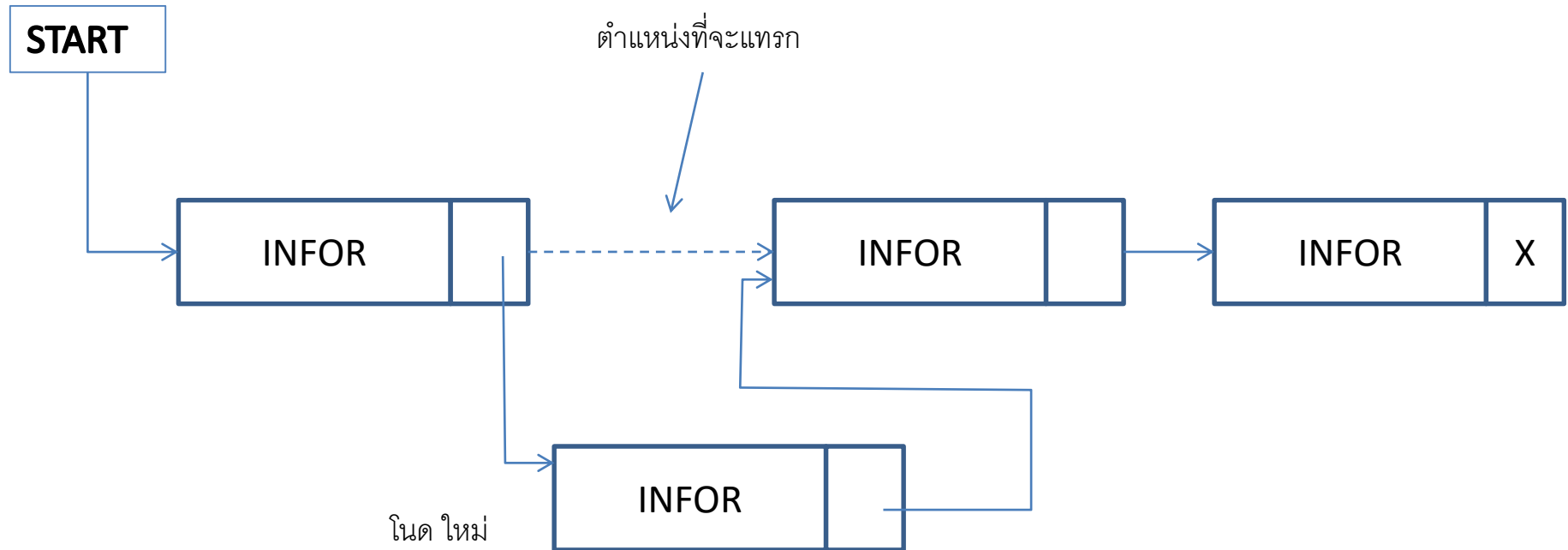
กิจกรรมนักศึกษา

- รันโปรแกรมทดสอบ
- เขียนโปรแกรมเพิ่มเติมตามโจทย์กำหนด

การแทรกข้อมูลลงในรายการโยงทางเดียว

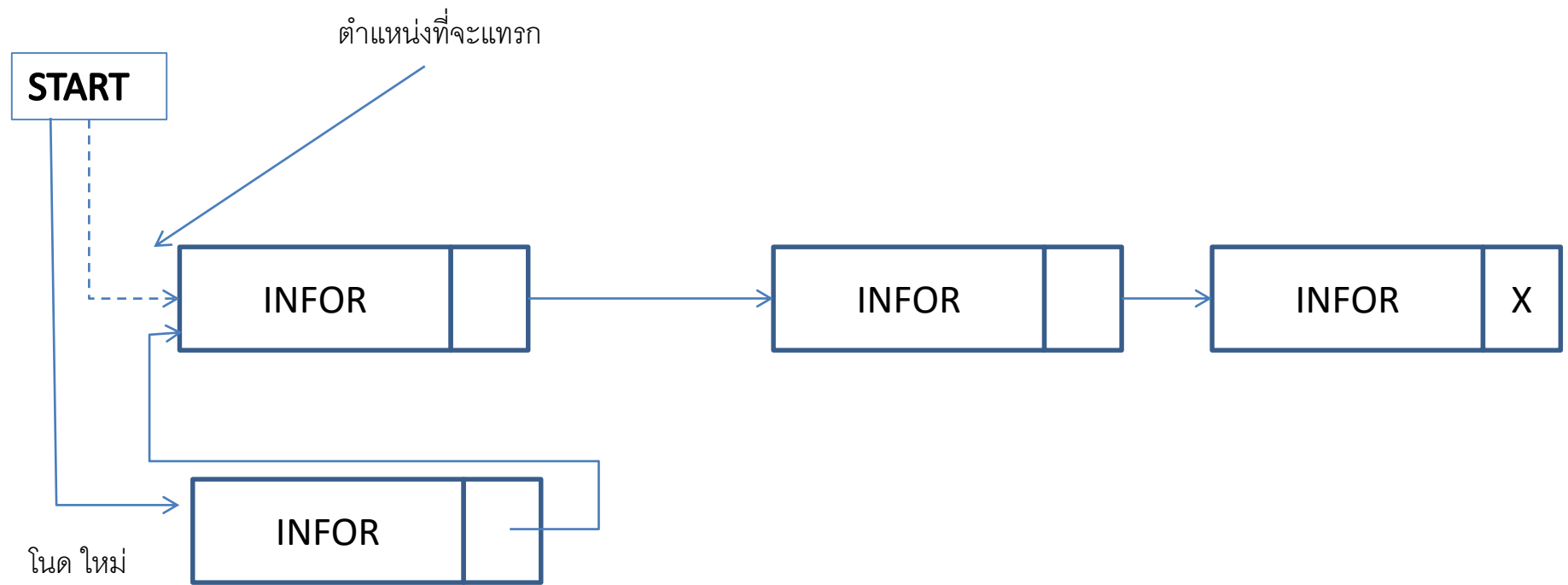
- การแทรก หมายถึง การนำโน้ตใหม่เพิ่มเข้าไปในรายการโยงเดิมอยู่แล้ว หรือหากยังไม่มี จะต้องทำให้โน้ตใหม่เป็นโน้ตแรกสุดของข้อมูลในลิงค์ลิสต์ การแทรกโน้ตเริ่มต้นจากสร้างโน้ตใหม่ แล้วระบุตำแหน่งที่ต้องการแทรก จากนั้น นำตัวชี้ของโน้ตใหม่ชี้ไปยังโน้ตที่อยู่หลังตำแหน่งหลังการแทรก ด้วยการพิจารณาจากตัวชี้โน้ตที่อยู่ก่อนหน้าตำแหน่งแทรก จากนั้น เปลี่ยนตัวชี้ที่เคยชี้ไปยังโน้ตถัดไปของตำแหน่งก่อนหน้าที่จะแทรก ให้มาชี้ยังโน้ตใหม่ แนวคิดเบื้องต้น

- การแทรกข้อมูลในลิสต์สามารถดำเนินการได้อย่างรวดเร็วโดย
 - เอาพอยเตอร์ของโนดใหม่ไปชี้ยังโนดที่ต้องการแทรกข้อมูลอยู่ เปลี่ยนพอยเตอร์ที่เคยชี้ไปยังโนดถัดไป ให้มาชี้ยังโนดใหม่

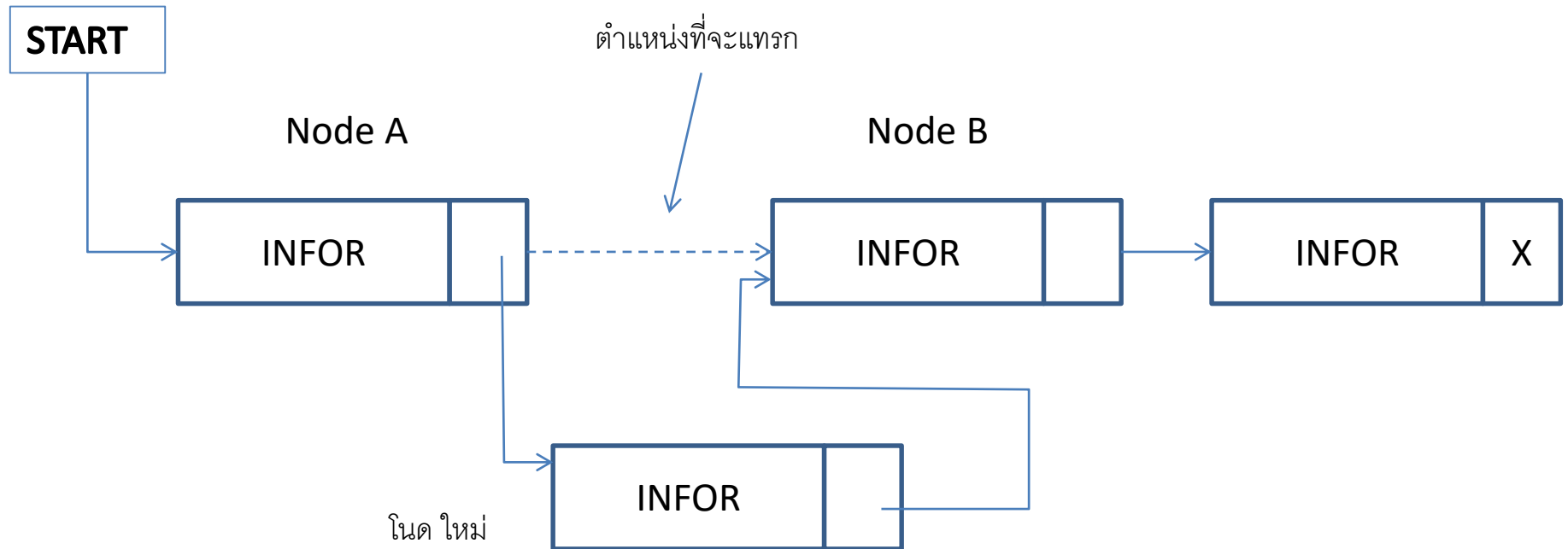


วิธีการแทรกข้อมูล

- แทรกที่ต้นลิสต์
- แทรกในตำแหน่งที่ต้องการ



แทรกยังตำแหน่งที่ที่ต้องการ

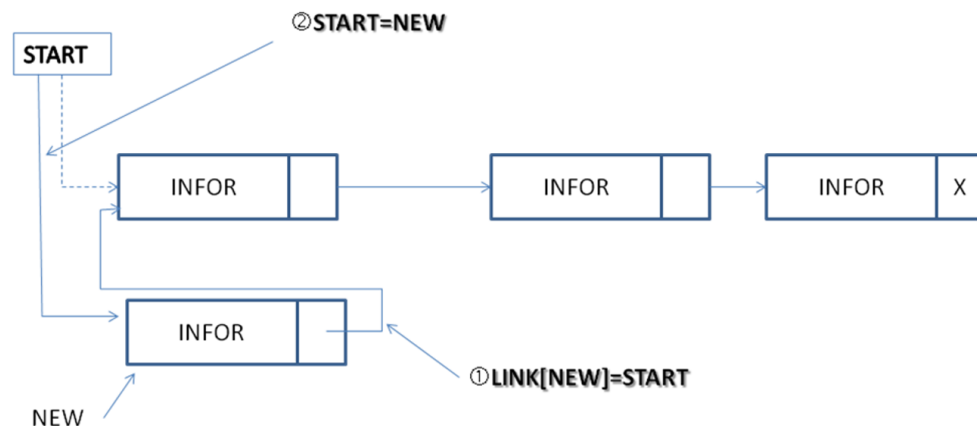


การแทรกข้อมูลลงในรายการโยงแบบต้นลิสต์

- การแทรกข้อมูลที่ต้นลิสต์ คือ การนำโนดเพิ่มเข้าไปเป็นรายการแรกของรายการโยง ขั้นตอนการดำเนินการไม่มีความซับซ้อน เพราะเพียงแค่สร้างโนดใหม่ขึ้นมา แล้วเอาไปตัวชี้ไปชี้โนดที่ **START** เคยชี้อยู่ จากนั้นเปลี่ยนตัวชี้ของ **START** มาชี้ที่โนดใหม่ เพื่อให้เข้าใจตัวแปรที่เกี่ยวข้องมากขึ้น จึงกำหนดนิยาม **6.4** เพื่ออธิบายอัลกอริทึมในลำดับถัดไป

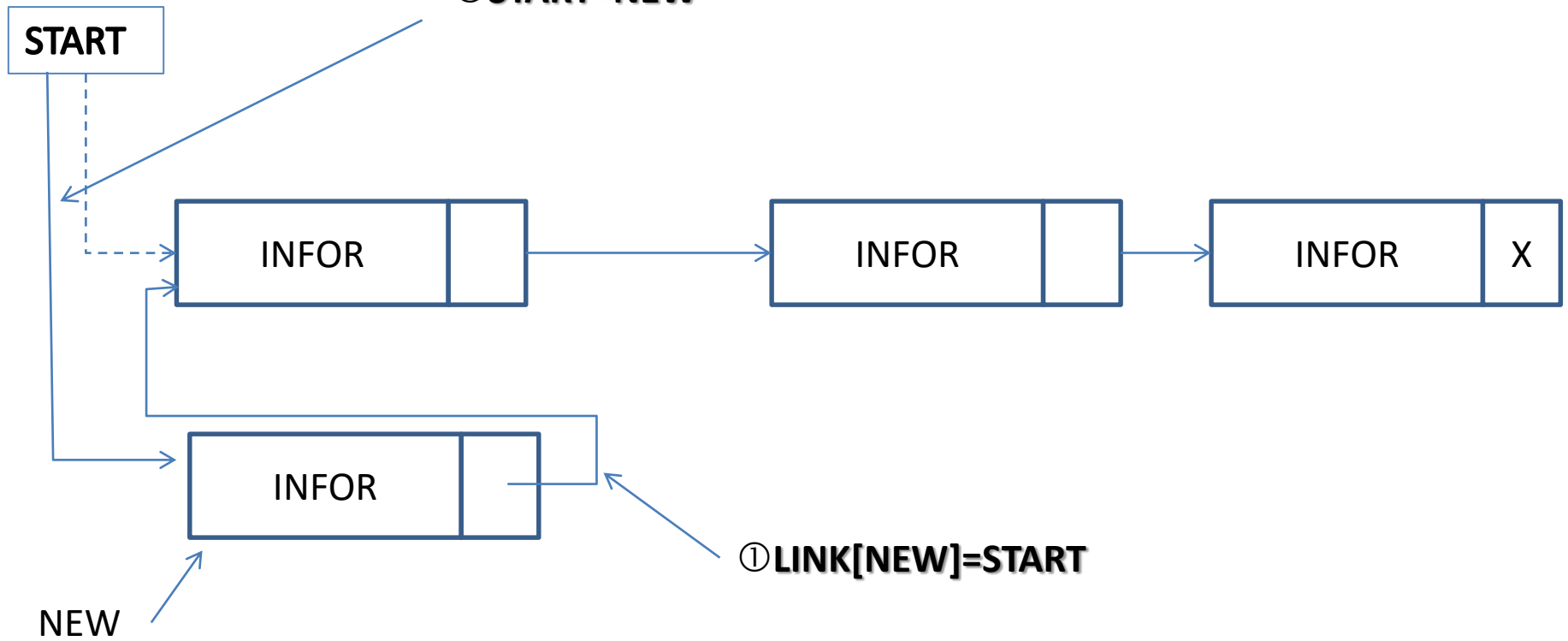
นิยามที่ 6.4 กำหนดให้ NEW คือ โหนดใดๆ ที่ถูกสร้างขึ้นมาใหม่ เพื่อใช้แทรกเข้าไปในรายการโยง

แสดงรูปตามแนวคิดของการแทรกโหนดใหม่ NEW เข้าไปในต้นรายการโยง ดังรูปที่ 6.25



รูปที่ 6.25 แสดงแนวคิดเริ่มต้นในการแทรก

② **START=NEW**



จากรูปอธิบายได้ว่า เบื้องต้นต้องสร้าง NEW ขึ้นมา และกำหนดข้อมูลที่ใส่ในส่วน INFOR ก่อน หลังจากนั้น นำเอาตัวชี้ของโนด NEW ซี่ไปยัง START แล้วเปลี่ยนตัวชี้ของ START มาชี้ที่ NEW ตามลำดับ แนวทางการแทรกและอัลกอริทึม ดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. NEW คือโนดใหม่
2. กำหนด $\text{INFOR}[\text{NEW}] = \text{ITEM}$
3. กำหนด $\text{LINK}[\text{NEW}] = \text{START}$
4. กำหนด $\text{START} = \text{NEW}$

Algorithm 6.4: InsertFirst

Input : LIST, START, NEW, ITEM

Output : ITEM is inserted into LIST at the first node.

1. $\text{INFOR}[\text{NEW}] := \text{ITEM}$
2. $\text{LINK}[\text{NEW}] := \text{START}$
3. $\text{START} := \text{NEW}$
4. Return



อิมพลีเมนต์

- public class Algor54INSTFIRST {
- static Node START = new Node();
-
- public static void main(String[] args) {
- START.INFOR = 10;
- System.out.println("Data Befor INSERT:");
- showDatainList();
- INSTFIRST(20);
- INSTFIRST(30);
- INSTFIRST(40);
- System.out.println("Data After INSERT:");
- showDatainList();
- }
-
- public static void INSTFIRST(int item){
- //Algorrithm 5.4 SEARCH if AVAIL = NULL step. 1
- Node NEW=new Node(); //step. 2
- int ITEM=item;
- NEW.INFOR = item;
- NEW.LINK = START;
- START=NEW;
- }
-
- public static void showDatainList(){
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- System.out.println(PTR.INFOR);
- PTR=PTR.LINK;
- }
- }
- }
- }

- public class Algor54INSFIRST {
- static Node START = new Node();
- public static void main(String[] args) {
 - START.INFOR = 10;
 - System.out.println("Data Befor INSERT:");
 - showDatainList();
 - INSTFIRST(20);
 - INSTFIRST(30);
 - INSTFIRST(40);
 - System.out.println("Data After INSERT:");
 - showDatainList();
- }
-

- public static void INSTFIRST(int item){
- //Algorithm 5.4 SEARCH if AVAIL = NULL step. 1
- Node NEW=new Node(); //step. 2
- int ITEM=item;
- NEW.INFOR = item;
- NEW.LINK = START;
- START=NEW;
- }
-

- public static void showDatainList(){
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- System.out.println(PTR.INFO);
- PTR=PTR.LINK;
- }

กิจกรรมนักศึกษา

- รันโปรแกรมทดสอบตามอัลกอริทึม
- สร้างลิสต์ของตนเองตามที่กำหนด

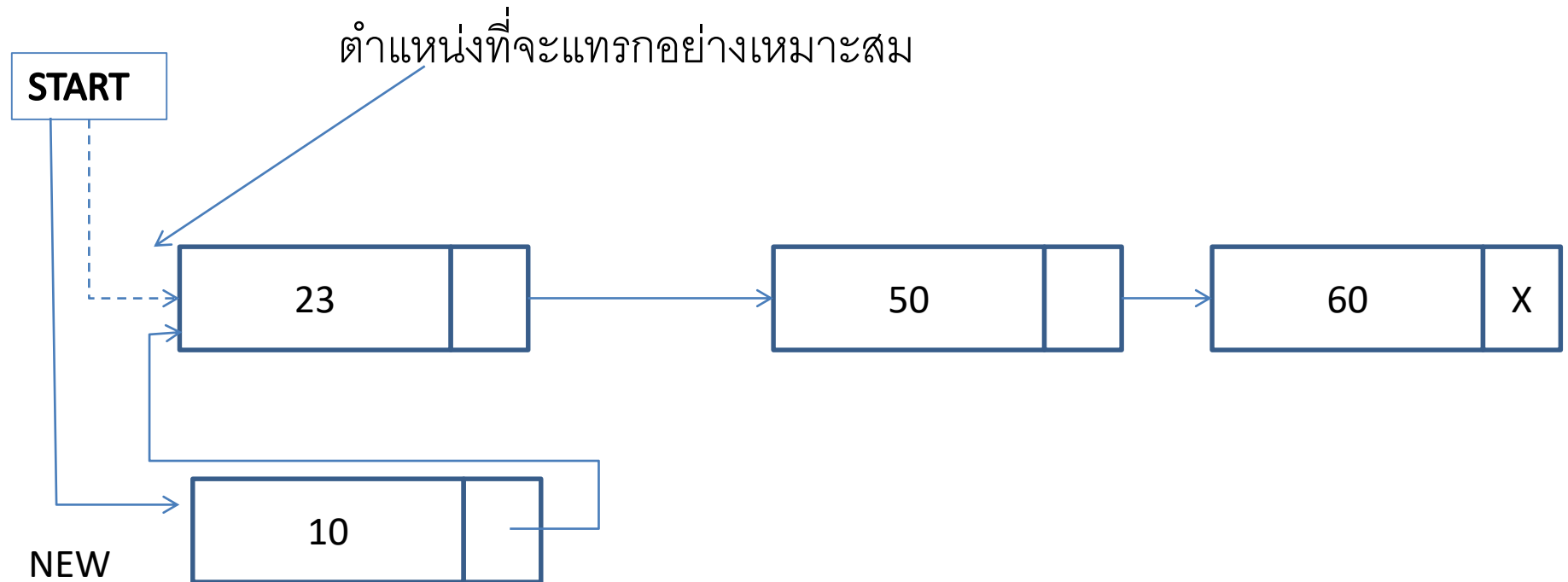
การแทรกในตำแหน่งที่ระบุโดยตรง

- แนวทางการแทรก คือ ต้องกำหนดโน้ตที่ต้องการแทรกก่อน จากนั้นกำหนดให้เป็น **LOC** ซึ่งตำแหน่งของโน้ตที่จะแทรกข้อมูลเข้าไปต่อท้าย แสดงแนวคิดเบื้องต้นเป็นรูปภาพได้ดังนี้

แทรกข้อมูลลงในลิสต์ตามตำแหน่งที่ต้องการ

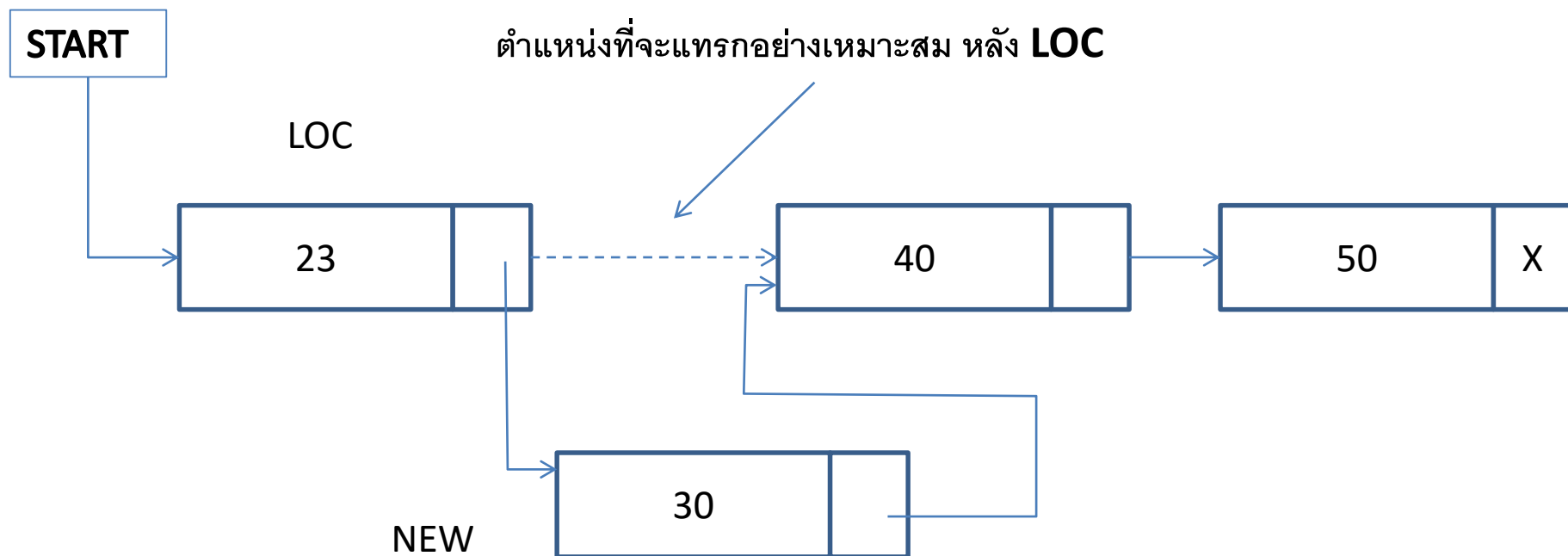
- แนวทางการแทรกคือ ต้องกำหนดโนดที่ต้องการแทรก ก่อน กำหนดให้เป็น LOC ซึ่งตำแหน่งของโนดที่จะแทรกข้อมูลเข้าไปต่อท้าย
- ตรวจสอบว่า **LOC=NULL** หรือไม่ หากเป็น **NULL**
 - แสดงว่า ให้แทรกต้นลิสต์
 - กำหนด **LINK[NEW]=START**
 - **START=NEW**
 - หากไม่เป็น **NULL**
 - กำหนด **LINK[NEW]=LINK[LOC]**
 - **LINK[LOC]=NEW**

เมื่อตำแหน่งเหมาะสมเป็นต้นลิสต์



LOC=NULL, LINK[NEW]=START, START=NEW

แทรกยังตำแหน่งที่ต้องการ



LINK[NEW]=LINK[LOC], LINK[LOC]=NEW

การแทรกโนดใหม่ ที่ตำแหน่งหลังโนด LOC ซึ่งอยู่ ต้องมีตัวแปร LIST, START, NEW, ITEM, LOC โดยที่ ITEM คือข้อมูลที่บรรจุในโนด NEW จะต้องถูกแทรกไปหลังโนดที่ LOC ซึ่งอยู่ ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. ตรวจสอบว่า LOC=NULL หรือไม่
หากเป็น NULL
2. แสดงว่า ให้แทรกต้นลิสต์
 - 2.1 กำหนด LINK[NEW]=START
 - 2.2 START=NEW
3. หากไม่เป็น NULL
4. กำหนด LINK[NEW]=LINK[LOC]
5. LINK[LOC]=NEW

Algorithm 6.5 : InsertLOC

Input : LIST, START, NEW, ITEM, LOC

Output : ITEM is inserted after LOC

```
1   INFOR[NEW] := ITEM
2   IF LOC = NULL Then
3       LINK[NEW] := START
4       START := NEW
5   Else
6       LINK[NEW] := LINK[LOC]
7       LINK[LOC] := NEW
8   End IF
9   Return
```



อิมพลีเมนต์

```

• public class Algor55INLOC {
•     static Node START = new Node();
•
•
•     public static void main(String[] args) {
•         START.INFOR = 100;
•         Node A = new Node(); A.INFOR = 90; START.LINK = A;
•         Node B = new Node(); B.INFOR = 80; A.LINK = B;
•         Node C = new Node(); C.INFOR = 70; B.LINK = C;
•         Node D = new Node(); D.INFOR = 60; C.LINK = D;
•         Node E = new Node(); E.INFOR = 50; D.LINK = E;
•         System.out.println("Data Befor INSERT:");
•         showDatainList();
•         INSTFIRST(70,65);
•         System.out.println("Data After INSERT:");
•         showDatainList();
•     }
•
•
•     public static void INSTLOC(int loc, int item){
•         //Algorithm 5.5 SEARCH if AVAIL = NULL step. 1
•
•         Node LOC=SEARCH(loc); //ค้นหาข้อมูลที่จะแทรก
•         Node NEW=new Node(); //step. 2
•         int ITEM=item;
•         NEW.INFOR = item;
•         if(LOC==null) {
•             NEW.LINK = START; //insert first Node
•             START=NEW;
•         }else
•         {
•             NEW.LINK = LOC.LINK;
•             LOC.LINK = NEW;
•         }
•     }
•
•     public static Node SEARCH(int ITEM){
•         Node LOC=null;
•         Node PTR=null;
•         PTR=START;
•         while(PTR!=null){
•             if(ITEM==(int)PTR.INFOR) {
•                 LOC = PTR;
•                 break;
•             }else
•                 PTR=PTR.LINK;
•         }
•         //Show Result;
•         if(LOC ==null)
•             System.out.println("Not found.");
•         else
•             System.out.println("Found.");
•
•         return LOC;
•     }
•
•
•     public static void showDatainList(){
•         Node PTR=null;
•         PTR=START;
•         while(PTR!=null){
•             System.out.println(PTR.INFOR);
•             PTR=PTR.LINK;
•         }
•     }
•

```

- public class Algor55INLOC {
- static Node START = new Node();
-
- public static void main(String[] args) {
- START.INFOR = 100;
- Node A = new Node(); A.INFOR = 90; START.LINK = A;
- Node B = new Node(); B.INFOR = 80; A.LINK = B;
- Node C = new Node(); C.INFOR = 70; B.LINK = C;
- Node D = new Node(); D.INFOR = 60; C.LINK = D;
- Node E = new Node(); E.INFOR = 50; D.LINK = E;
- System.out.println("Data Befor INSERT:");
- showDatainList();
- INSTFIRST(70,65);
- System.out.println("Data After INSERT:");
- showDatainList();
- }
-

- public static void INSTLOC(int loc, int item){
- //Algorithm 5.5 SEARCH if AVAIL = NULL step. 1
-
- Node LOC=SEARCH(loc); //ค้นหาข้อมูลที่จะแทรก
- Node NEW=new Node(); //step. 2
- int ITEM=item;
- NEW.INFOR = item;
- if(LOC==null) {
- NEW.LINK = START; //insert first Node
- START=NEW;
- }else
- {
- NEW.LINK = LOC.LINK;
- LOC.LINK = NEW;
- }
- }



- public static Node SEARCH(int ITEM){
- Node LOC=null;
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- if(ITEM==(int)PTR.INFOR) {
- LOC = PTR;
- break;
- }else
- PTR=PTR.LINK;
- }
- //Show Result;
- if(LOC ==null)
- System.out.println("Not found.");
- else
- System.out.println("Found.");
-
- return LOC;
- }



- `public static void showDatainList(){`
- `Node PTR=null;`
- `PTR=START;`
- `while(PTR!=null){`
- `System.out.println(PTR.INFOR);`
- `PTR=PTR.LINK;`
- `}`
-
- `}`

กิจกรรมนักศึกษา

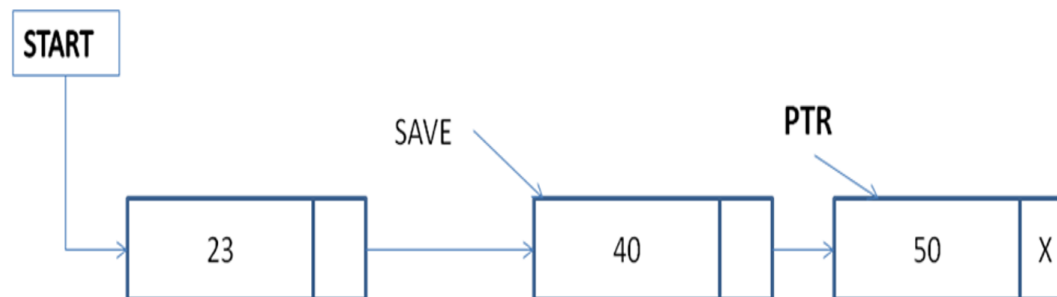
- รันโปรแกรมทดสอบตามอัลกอริทึมตัวอย่าง
- ทดลองเขียนโปรแกรมตามที่กำหนด

การแทรกโนดในลิสต์ที่มีการจัดเรียงแล้ว

- แนวทางการแทรก ต้องค้นหาตำแหน่งที่จะแทรกให้พบก่อน **LOC** สำหรับการค้นหาจะต้องค้นเพื่อให้รู้ว่า ตำแหน่งที่จะแทรกอยู่ระหว่างข้อมูลสองโนด ดังนั้น จำเป็นจะต้องใช้ตัวชี้สองตัวดำเนินการค้นหาไปพร้อมๆ กัน เมื่อตัวชี้ตัวหน้า (**LOC**) ที่ตำแหน่งใด จะเก็บตำแหน่งที่ตัวชี้ตัวหลังชี้อยู่เป็นค่า **LOC** เพื่อใช้งานสำหรับการแทรก ดังนั้น จึงจำเป็นต้องมีตัวแปรสำหรับตัวชี้เพิ่มขึ้นอีกหนึ่งตัว ตามนิยามที่ 6.5

นิยามที่ 6.5 กำหนดให้ SAVE คือ ตัวชี้สำหรับชี้ไปยังโนดของรายการโยงใดๆ

ดังนั้น การค้นหาต้องตัวชี้สองตัว PTR และ SAVE ค้นหาในลิสต์ เมื่อค้นที่ตำแหน่งที่ PTR ชี้อยู่ แสดงว่าตำแหน่งที่จะแทรกจะอยู่ระหว่าง SAVE กับ PTR ให้กำหนด $LOC=SAVE$ เพื่อคืนค่า LOC จากนั้น เรียกใช้งานอัลกอริทึมการแทรก ณ ตำแหน่งที่ต้องการให้ทำงาน ค้นหาตำแหน่งที่ต้องการ แสดงตัวอย่าง การค้นโดยใช้ตัวชี้สองตัวดังรูปที่ 6.27 เมื่อ $ITEM=45$ ตำแหน่งที่เหมาะสมระหว่าง 40 กับ 50



Algorithm 6.6 FINDLOC**Input** : LIST, START, SAVE, ITEM, LOC**Output** : LOC

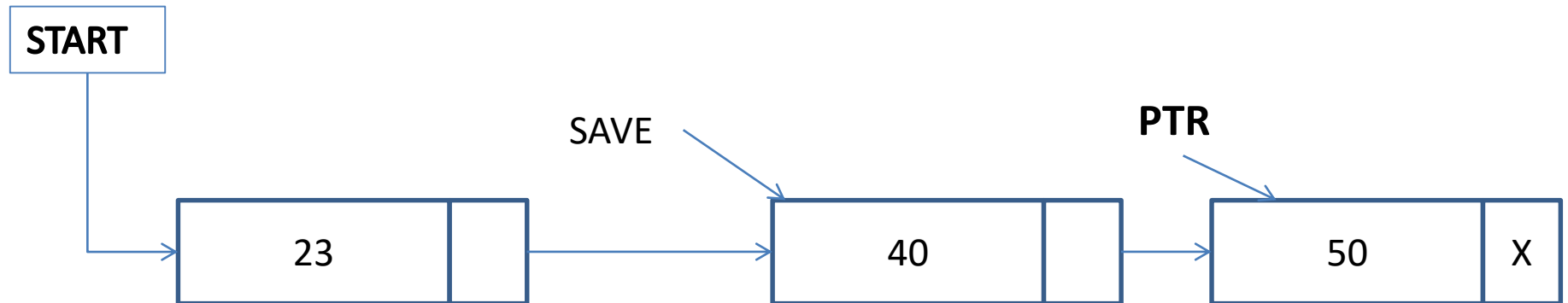
```
1.   IF START=NULL Then
2.       LOC := NULL
3.       Return LOC
4.   End IF
5.   IF ITEM< INFOR[START] Then
6.       LOC := NULL
7.       Return LOC
8.   End IF
9.   SAVE := START, PTR := LINK[START]
10.  While PTR!=NULL Do
11.      IF ITEM< INFOR[PTR] Then
12.          LOC := SAVE
13.          Return LOC
14.      End IF
15.      SAVE := PTR;
16.      PTR := LINK[PTR]
17.  End While
18.  LOC := SAVE
19.  Return LOC
```

การแทรกโนดในลิสต์ที่มีการจัดเรียงไว้แล้ว

- ต้องค้นหาตำแหน่งที่จะแทรกให้พบก่อน LOC
 - โดยการใช้พอยเตอร์สองตัวค้นหาเข้าไปในลิสต์ PTR, SAVE
 - เมื่อค้นเข้าไปจนพบ ตำแหน่งที่ต้องการกำหนด $LOC=SAVE$ เพื่อคืนค่า LOC
- จากนั้นเรียกใช้งานอัลกอริทึมการแทรก ณ ตำแหน่งที่ต้องการให้ทำงาน

ค้นหาตำแหน่งที่ต้องการ

- ถ้า $ITEM=45$ ตำแหน่งที่เหมาะสมระหว่าง 40 กับ 50



ตำแหน่งที่จะแทรกต่อท้าย **LOC=SAVE**

- Algorithm Find LOC in a sorted Link Lists
- Input : LIST, START, SAVE, ITEM, LOC
- Output : LOC
- IF START=NULL Then
- LOC=NULL
- Return LOC
- End IF
- IF ITEM< INFOR[START] Then
- LOC=NULL
- Return LOC
- End IF
- SAVE=START, PTR=LINK[START]
- While PTR!=NULL Do
- IF ITEM< INFOR[PTR] Then
- LOC = SAVE;
- Return LOC;
- End IF
- SAVE=PTR;
- PTR=LINK[PTR]
- End While
- LOC=SAVE;
- Return LOC

- Algorithm Find LOC in a sorted Link Lists

- Input : LIST, START, SAVE, ITEM, LOC

- Output : LOC

- IF START=NULL Then

- LOC=NULL

- Return LOC

- End IF

- IF ITEM< INFOR[START] Then

- LOC=NULL

- Return LOC

- End IF

- SAVE=START, PTR=LINK[START]

- While PTR!=NULL Do

- IF ITEM< INFOR[PTR] Then

- LOC = SAVE;

- Return LOC;

- End IF

- SAVE=PTR;

- PTR=LINK[PTR]

- End While

- LOC=SAVE;

- Return LOC

กรณีไม่มีข้อมูลลิสต์

กรณีข้อมูลที่คั่นน้อยกว่าโนดแรก

อัลกอริทึมแทรกตำแหน่งที่ต้องการ (ข้อมูลเรียงแล้ว)

- เรียก Find LOC in a sorted Link Lists ให้ทำงาน
- เรียก Insert Item at LOC in Link Lists ให้ทำงาน



อิมพลีเมนต์

```

package linklistimplementation;

/**
 *
 * @author ComSCiv3400
 */
public class Algor56TO57FINDA {
    static Node START = new Node();

    public static void main(String[] args) {
        START.INFOR = 50;
        Node A = new Node(); A.INFOR = 60; START.LINK = A;
        Node B = new Node(); B.INFOR = 70; A.LINK = B;
        Node C = new Node(); C.INFOR = 80; B.LINK = C;
        Node D = new Node(); D.INFOR = 90; C.LINK = D;
        Node E = new Node(); E.INFOR = 100; D.LINK = E;
        System.out.println("Data Befor INSERT:");
        showDataInList();
        Node LOC=FINDA(70); //ค้นหาข้อมูลที่จะแทรก //call FINDA(...)
        INSLOC(LOC,75); //Call INSLOC(...)
        System.out.println("Data After INSERT:");
        showDataInList();
    }

    public static void INSLOC(Node LOC, int item){
        //Algorithm 5.5 SEARCH if AVAIL = NULL step. 1
        Node NEW=new Node(); //step. 2
        int ITEM=item;
        NEW.INFOR = item;
        if(LOC==null) {
            NEW.LINK = START; //insert first Node
            START=NEW;
        }else
        {
            NEW.LINK = LOC.LINK;
            LOC.LINK = NEW;
        }
    }

    public static Node FINDA(int ITEM){
        Node LOC=null;
        Node PTR=null;
        Node SAVE=null;
        PTR=START;
        if(START==null) { LOC=null; return LOC;}
        if(ITEM<(int)PTR.INFOR) { LOC=null; return LOC; }
        SAVE=START; PTR=START.LINK;
        while(PTR!=null){
            if(ITEM<(int)PTR.INFOR) {
                LOC = SAVE;
                return LOC;
            }
            SAVE=PTR;
            PTR=PTR.LINK;
        }
        LOC=SAVE;
        //Show Result;
        if(LOC ==null)
            System.out.println("Not found.");
        else
            System.out.println("Found.");
    }
}

```

- public class Algor56TO57FINDA {
- static Node START = new Node();
-
- public static void main(String[] args) {
- START.INFOR = 50;
- Node A = new Node(); A.INFOR = 60; START.LINK = A;
- Node B = new Node(); B.INFOR = 70; A.LINK = B;
- Node C = new Node(); C.INFOR = 80; B.LINK = C;
- Node D = new Node(); D.INFOR = 90; C.LINK = D;
- Node E = new Node(); E.INFOR = 100; D.LINK = E;
- System.out.println("Data Befor INSERT:");
- showDatainList();
- Node LOC=FINDA(70); //ค้นหาข้อมูลที่จะแทรก //call FINDA(...)
- INSLOC(LOC,75); //Call INSLOC()...
- System.out.println("Data After INSERT:");
- showDatainList();
- }

- public static void INSLOC(Node LOC, int item){
- //Algorithm 5.5 SEARCH if AVAIL = NULL step. 1
- Node NEW=new Node(); //step. 2
- int ITEM=item;
- NEW.INFOR = item;
- if(LOC==null) {
- NEW.LINK = START; //insert first Node
- START=NEW;
- }else
- {
- NEW.LINK = LOC.LINK;
- LOC.LINK = NEW;
- }
- }

- public static Node FINDA(int ITEM){
- Node LOC=null;
- Node PTR=null;
- Node SAVE=null;
- PTR=START;
- if(START==null) { LOC=null; return LOC;}
- if(ITEM<(int)PTR.INFOR) { LOC=null; return LOC; }
- SAVE=START; PTR=START.LINK;
- while(PTR!=null){
- if(ITEM<(int)PTR.INFOR) {
- LOC = SAVE;
- return LOC;
- }
- SAVE=PTR;
- PTR=PTR.LINK;
- }
- LOC=SAVE;
- //Show Result;
- if(LOC ==null)
- System.out.println("Not found.");
- else
- System.out.println("Found.");
-
- return LOC;
- }

กิจกรรมนักศึกษา

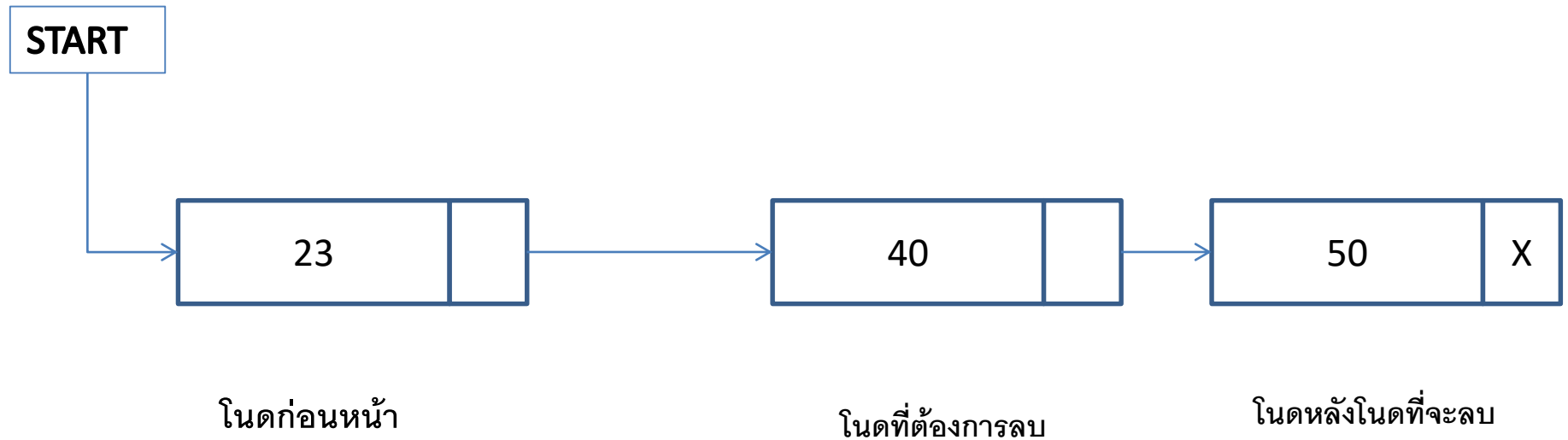
- รันโปรแกรมทดสอบตามอัลกอริทึม
- ทดลองเขียนโปรแกรมตามที่โจทย์กำหนด

การลบข้อมูลออกจากรายการโยงทางเดียว

- การลบข้อมูลในรายการโยง หมายถึง การลบโน้ตที่ต้องการออกจากรายการโยง สามารถทำได้โดยง่ายและรวดเร็ว ซึ่งถือว่าเป็นคุณสมบัติเด่นประการหนึ่งของ รายการโยง แนวคิดในการลบ จะต้องกำหนดโน้ตที่จะลบโดยนำโน้ตก่อนโน้ตที่จะลบ และโน้ตหลังจากโน้ตที่ต้องการจะลบด้วย กระบวนการลบทำได้โดยนำตัวชี้โน้ตก่อนหน้าโน้ตที่จะลบ ชี้ข้ามไปยังตัวหลังโน้ตที่ลบ จะทำให้โน้ตที่ต้องการไม่สามารถเข้าถึงได้ เป็นการลบไปโดยปริยาย จากนั้นอาจคั่นค่าหน่วยความจำให้กับระบบ เช่น ในภาษาซีพลัสพลัสอาจต้องคั่นหน่วยความจำ แต่ในภาษาจาวาไม่ต้องคั่นเพราะมีระบบจัดการหน่วยความจำอยู่แล้ว เป็นต้น แสดงแนวคิดการลบ ดังรูป 6.28

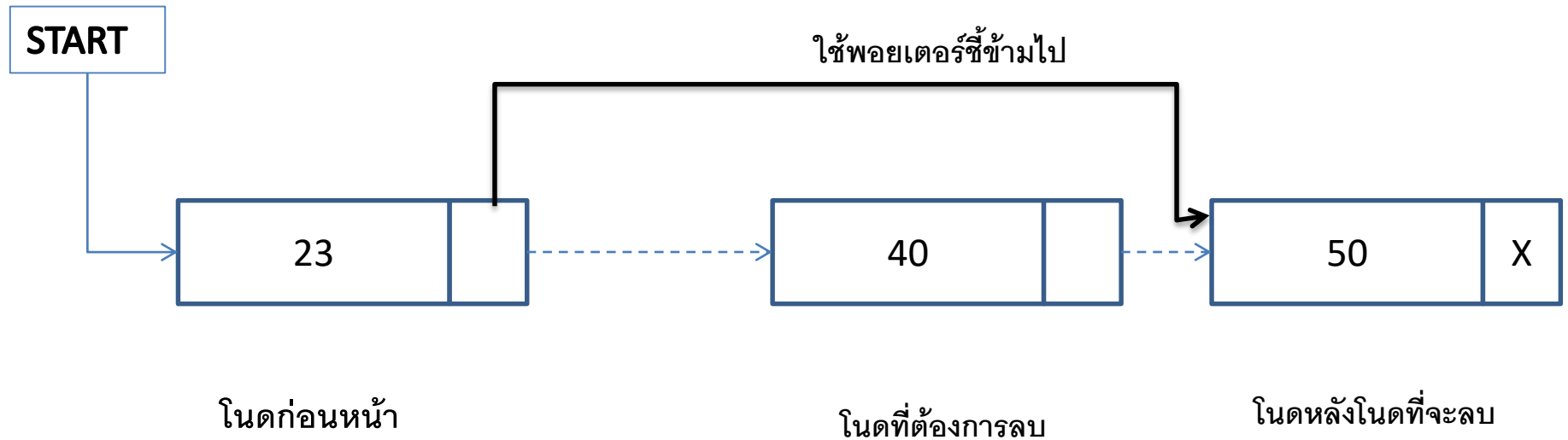
แสดงแนวความคิดการลบ

- ก่อนลบ



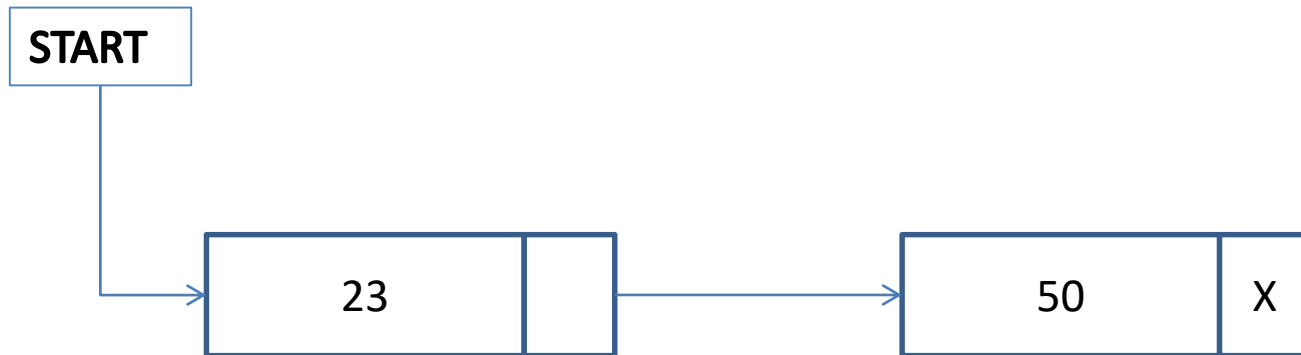
แสดงแนวความคิดการลบ

- ขั้นตอนลบ

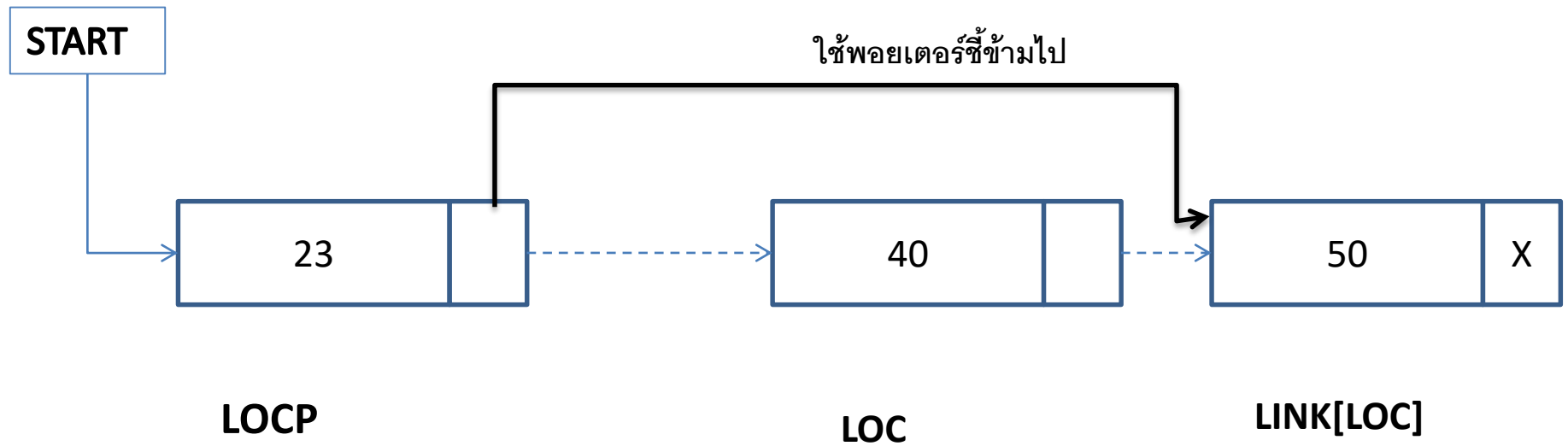


แสดงแนวความคิดการลบ

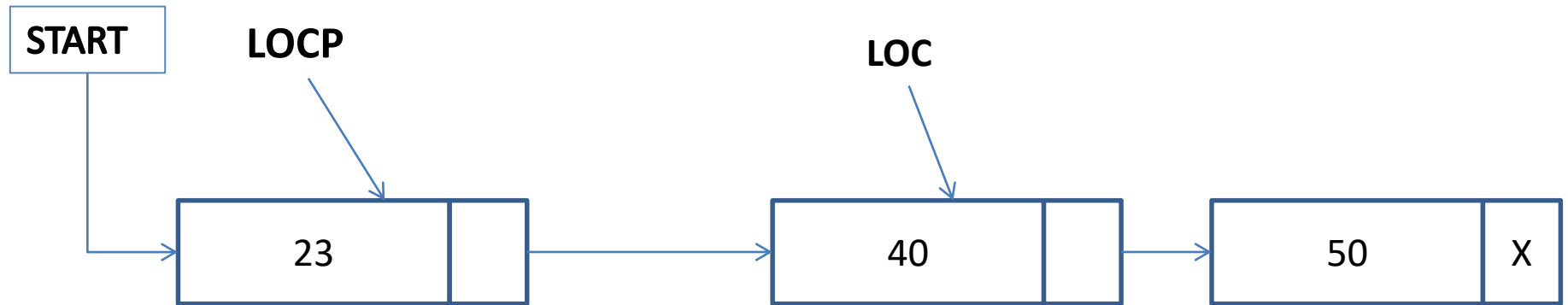
- หลังลบ



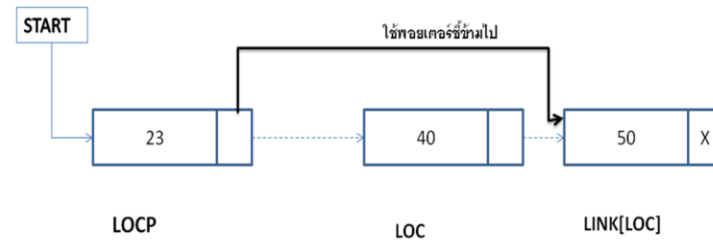
ตัวแปรที่เกี่ยวข้องกับการลบ



ก่อนการลบจะต้องระบุ LOCP, LOC



ดังที่กล่าวไปแล้วว่า การลบข้อมูลจำเป็นต้องจดจำตำแหน่งโนดก่อนหน้า โนดที่จะลบ และโนดหลังโนดที่จะลบ ต่อไปนี้จะใช้ตัวแปร LOCP แสดงตำแหน่งก่อนหน้าโนดที่จะลบ LOC แทนตำแหน่งของโนดที่จะลบ ขณะที่ LINK[LOC] แทนตำแหน่งหลังจากโนดที่จะลบ แสดงแนวคิดดังกล่าว ดังรูปที่ 6.29



รูปที่ 6.29 แสดงแนวคิดและตัวแปรดำเนินการก่อนการลบ

ก่อนการลบจะต้องระบุ LOCP LOC ทั้งนี้ อาจได้มาจากการค้นหาโดยใช้อัลกอริทึมสำหรับค้นหาข้อมูลเสียก่อน แสดงตำแหน่งตัวชี้ก่อนการลบข้อมูล ดังรูปต่อไปนี้

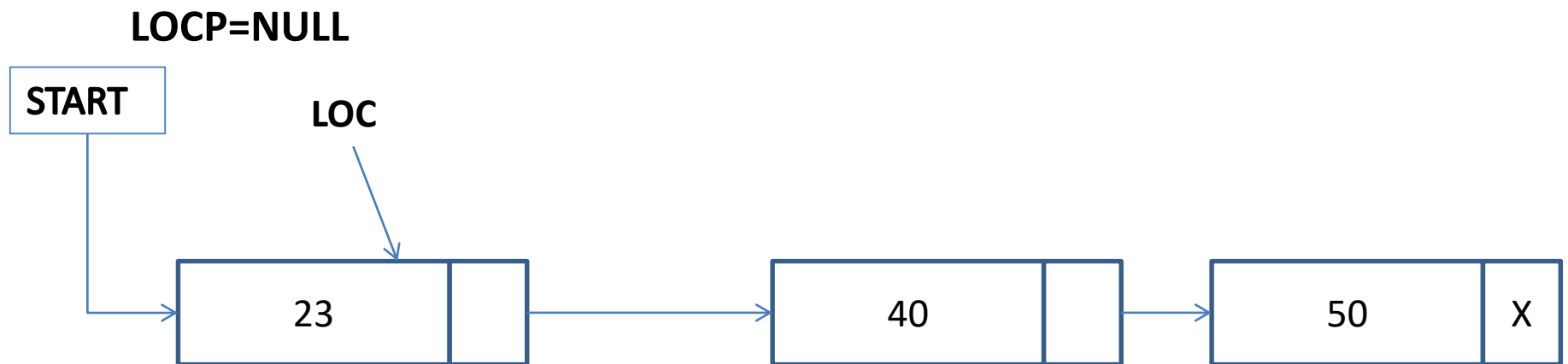


รูปที่ 6.30 แสดงการระบุตัวแปรก่อนการลบ

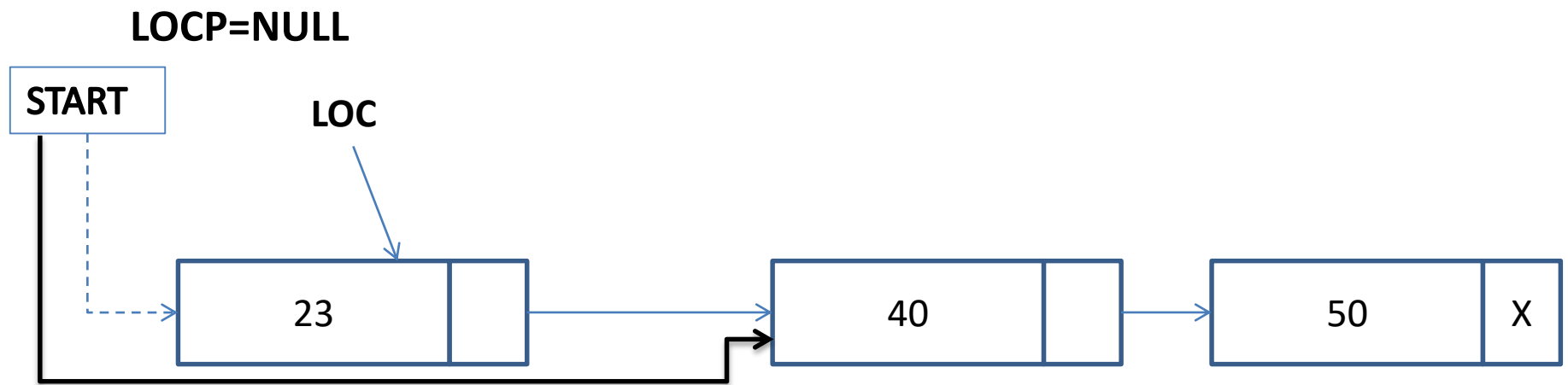
การลบเกิดขึ้น 2 กรณี

- ลบโนดแรก
 - $LOCP = NULL, LOC = START$
- ลบโนดอื่นๆ
 - $LOCP, LOC$ ไม่เท่ากับ $NULL$

LOCP=NULL, LOC=START (ก่อนลบ)

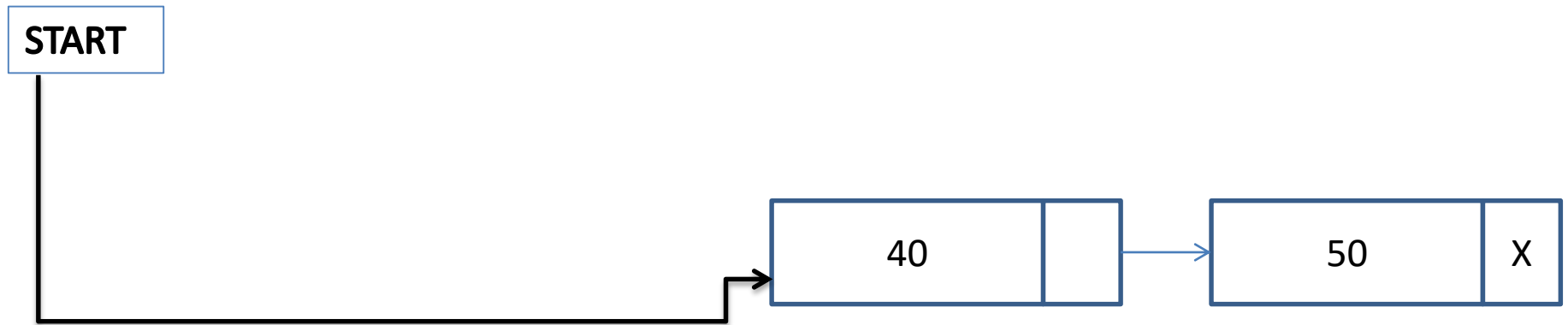


LOCP=NULL, LOC=START (ขณะลบ)

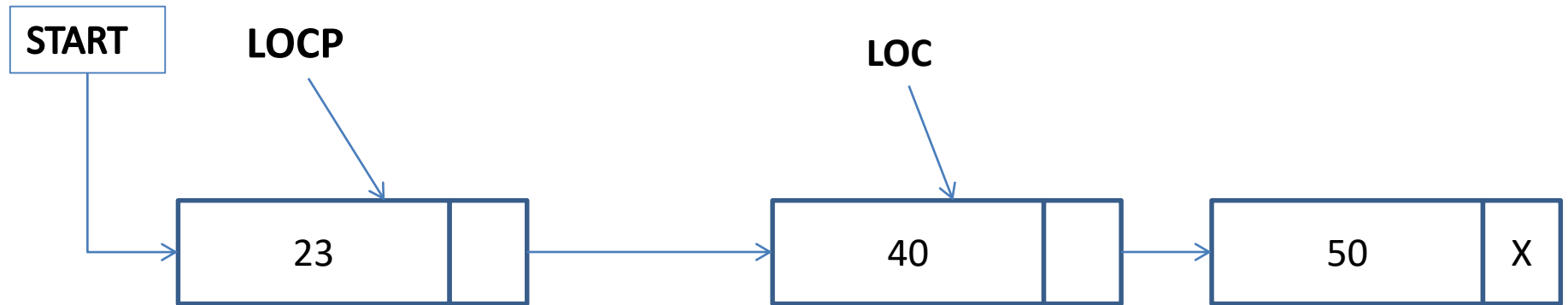


START=LINK[START]

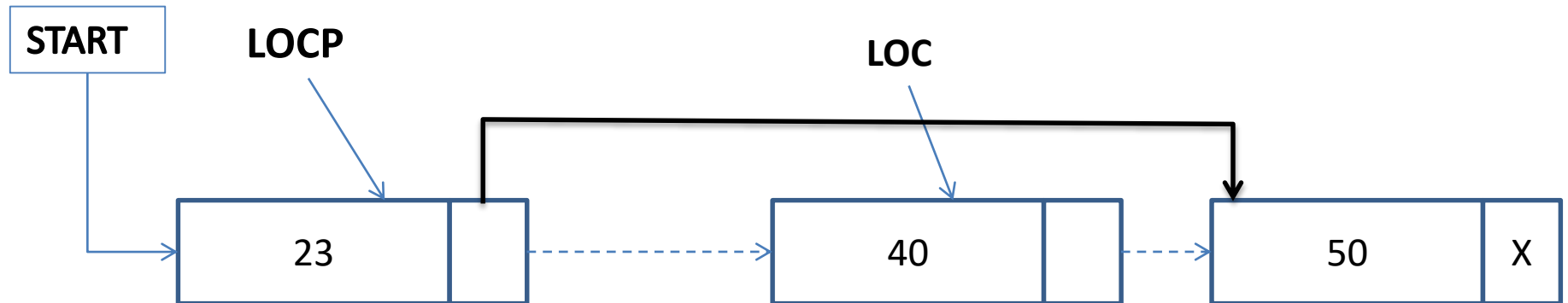
LOCP=NULL, LOC=START (หลังลบ)



LOCP, LOC ไม่เป็น NULL(ก่อนลบ)

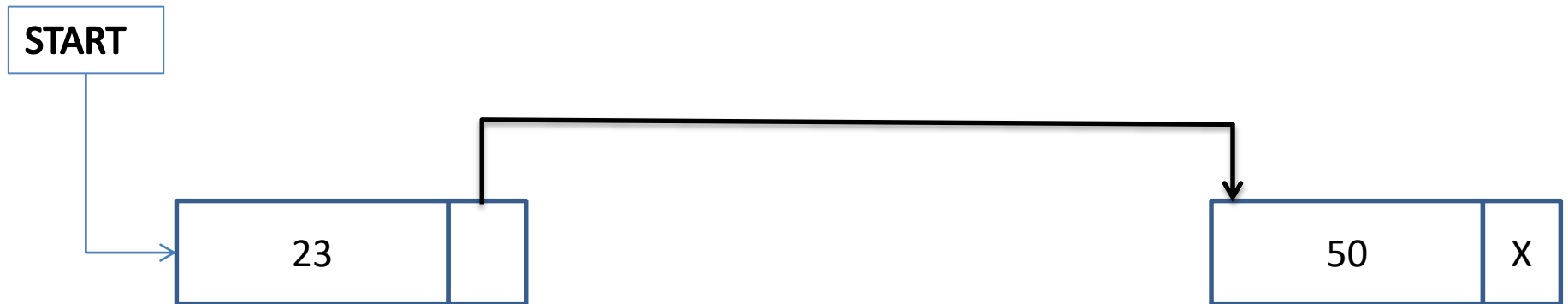


LOCP, LOC ไม่เป็น NULL (ขณละบ)



$$\text{LINK}[\text{LOCP}] = \text{LINK}[\text{LOC}]$$

LOCP, LOC ไม่เป็น NULL(หลังลบ)



แนวคิดการลบโนดที่ LOC ซ้อยู่ออกจากลิสต์ มีตัวแปร LIST, INFOR, LINK, START, LOCP, LOC โดยที่โนดที่ LOC ซ้อยู่จะถูกลบออกไปจาก LIST ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. ตรวจสอบว่า ค่า LOC เป็น NULL หรือไม่หากเป็น NULL ให้ลบที่ โหนดแรก โดย กำหนด $START = LINK[START]$
2. หากไม่เป็น LOC ไม่เป็น NULL ให้ดำเนินการลบที่โนด ที่ LOC ซ้อยู่ โดยกำหนด $LINK[LOCP] = LINK[LOC]$

Algorithm 6.7 : DELITEM

Input : LIST, INFOR, LINK, START, LOCP, LOC

Output : Node LOC is deleted.

1. IF LOC = NULL Then
2. $START := LINK[START]$
3. Else
4. $LINK[LOCP] := LINK[LOC]$
5. End IF
6. Return



อิมพลีเมนต์

```

• package linklistimplementation;

•
• /**
•  *
•  * @author ComSCiv3400
•  */
• public class Algor58DEL {
•     static Node START = new Node();
•
•
•     public static void main(String[] args) {
•         START.INFOR = 50;
•         Node A = new Node(); A.INFOR = 60; START.LINK = A;
•         Node B = new Node(); B.INFOR = 70; A.LINK = B;
•         Node C = new Node(); C.INFOR = 80; B.LINK = C;
•         Node D = new Node(); D.INFOR = 90; C.LINK = D;
•         Node E = new Node(); E.INFOR = 100; D.LINK = E;
•         Node LOCP=B;
•         Node LOC=C;
•         System.out.println("Data Befor INSERT:");
•         showDatainList();
•         DEL(LOCP,LOC, START);    //Call DEL()...
•         System.out.println("Data After INSERT:");
•         showDatainList();
•     }
•
•
•     public static void DEL(Node LOCP, Node LOC, Node START){
•         //Algorrithm 5.8 Delete
•         if(LOCP==null) {
•             START = START.LINK; //insert first Node
•         }else
•         {
•             LOCP.LINK = LOC.LINK;
•         }
•     }
•
•
•
•     public static void showDatainList(){
•         Node PTR=null;
•         PTR=START;
•         while(PTR!=null){
•             System.out.println(PTR.INFOR);
•             PTR=PTR.LINK;
•         }
•     }
• }

```

- public class Algor58DEL {
- static Node START = new Node();
-
- public static void main(String[] args) {
- START.INFOR = 50;
- Node A = new Node(); A.INFOR = 60; START.LINK = A;
- Node B = new Node(); B.INFOR = 70; A.LINK = B;
- Node C = new Node(); C.INFOR = 80; B.LINK = C;
- Node D = new Node(); D.INFOR = 90; C.LINK = D;
- Node E = new Node(); E.INFOR = 100; D.LINK = E;
- Node LOCP=B;
- Node LOC=C;
- System.out.println("Data Befor INSERT:");
- showDatainList();
- DEL(LOCP,LOC, START); //Call DEL()...
- System.out.println("Data After INSERT:");
- showDatainList();
- }

- public static void DEL(Node LOCP, Node LOC, Node START){
- //Algorrithm 5.8 Delete
- if(LOCP==null) {
- START = START.LINK; //insert first Node
- }else
- {
- LOCP.LINK = LOC.LINK;
- }
- }

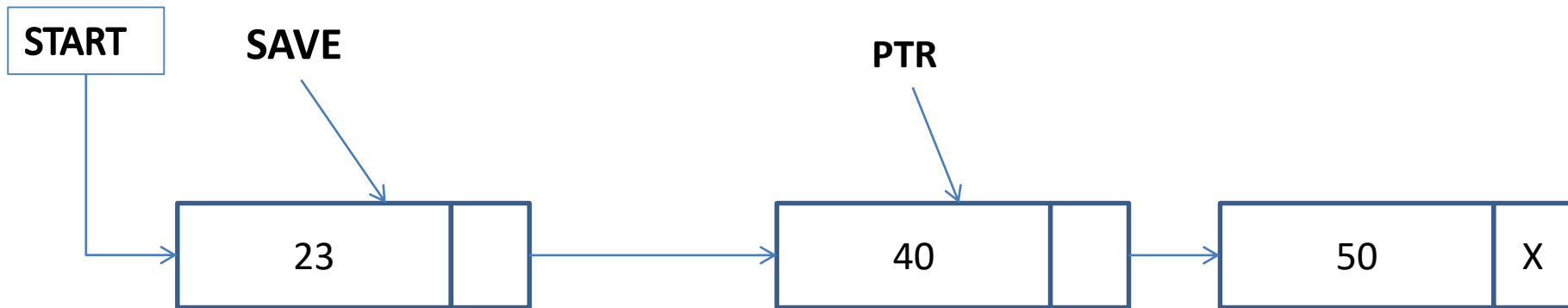
-
- public static void showDatainList(){
- Node PTR=null;
- PTR=START;
- while(PTR!=null){
- System.out.println(PTR.INFOR);
- PTR=PTR.LINK;
- }
-
- }

กิจกรรมนักศึกษา

- รันโปรแกรมทดสอบ
- ทดลองเขียนโปรแกรมลบในคตามที่ต้องการ

การหาตำแหน่ง LOCP, LOC

- ใช้พอยเตอร์ **SAVE** เพื่อท่องและคืนค่าเป็น **LOCP**
- ใช้พอยเตอร์ **PTR** เพื่อท่องและคืนค่าเป็น **LOC**



- Algorithm FINB Find Item in a Link Lists
- Input : LIST, INFOR, LINK, START, LOCP, LOC, SAVE, PTR, ITEM
- Output : LOCP, LOC.
- IF START = NULL Then
- LOCP= LOC=NULL
- Return LOCP, LOC
- End IF
- IF INFOR[START]=ITEM Then
- LOCP=NULL,
- LOC = START
- Return LOCP, LOC
- End IF
- SAVE=START, PTR=LINK[START]
- While PTR!=NULL DO
- IF INFOR[PTR]=ITEM Then
- LOCP=SAVE, LOC=PTR
- Return LOCP, LOC
- End IF
- SAVE=PTR, PTR=LINK[PTR]
- End While
- End IF
- Exit

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. ตรวจสอบว่า เป็นลิสต์ว่างหรือไม่
หากเป็นลิสต์ว่า ให้กำหนด
LOCP, LOC เป็น NULL แล้วคือ
ค่า จบงาน
2. ตรวจสอบว่า ค่าข้อมูลโนดแรกสุด
เท่ากับข้อมูลที่ต้องการค้นหา
หรือไม่ หากเท่ากัน ให้กำหนด
LOCP เป็น NULL ส่วน LOC ให้
เป็น START
3. ถ้าไม่ใช่สองกรณีดังกล่าว ให้
กำหนด การทอ้งโดยใช้ตัวชี้
SAVE ชี้ที่ START และตัวชี้ PTR
ชี้ตัวที่ START ช้อยู่ แล้ววน
เปรียบเทียบจนพบข้อมูลตรงกับ
ITEM จึง ก า ห น ด ให้
LOCP=SAVE LOC=PTR แล้ว
คืนค่า LOCP และ LOC

Algorithm 6.8 : FINDLOCPandLOC

Input : LIST, INFOR, LINK, START, LOCP,
LOC, SAVE, PTR, ITEM

Output : LOCP, LOC.

1. IF START = NULL Then
2. LOCP := LOC := NULL
3. Return LOCP, LOC
4. End IF
5. IF INFOR[START]=ITEM Then
6. LOCP := NULL,
7. LOC := START
8. Return LOCP, LOC
9. End IF
10. SAVE := START, PTR := LINK[START]
11. While PTR!=NULL DO
12. IF INFOR[PTR]=ITEM Then
13. LOCP := SAVE, LOC := PTR
14. Return LOCP, LOC
15. End IF
16. SAVE := PTR, PTR := LINK[PTR]
17. End While
18. End IF
19. Return



อิมพลีเมนต์

```

• package linklistimplementation;

•
• /**
•  *
•  * @author ComSCiv3400
•  */
• public class Algor59to10TFINDB {
•     static Node START = new Node();
•     static Node LOCP=null;
•     static Node LOC=null;
•     public static void main(String[] args) {
•         START.INFOR = 50;
•         Node A = new Node(); A.INFOR = 60; START.LINK = A;
•         Node B = new Node(); B.INFOR = 70; A.LINK = B;
•         Node C = new Node(); C.INFOR = 80; B.LINK = C;
•         Node D = new Node(); D.INFOR = 90; C.LINK = D;
•         Node E = new Node(); E.INFOR = 100; D.LINK =E;
•         System.out.println("Data Befor INSERT.");
•         showDatainList();
•         DEL(70);      //Call INSLOC()...
•         System.out.println("Data After INSERT.");
•         showDatainList();
•     }

•
• public static void DEL(int ITEM){
•     //Algorithm 5.8 Delete
•     FINDB(ITEM);
•     if(LOCP==null) {
•         START = START.LINK; //insert first Node
•     }else
•     {
•         LOCP.LINK = LOC.LINK;
•     }
• }

•
• public static boolean FINDB(int ITEM){
•
•     Node PTR=null;
•     Node SAVE=null;
•     PTR=START;
•     if(START==null) { LOC=null; LOCP=null; return true;}
•     if((int)START.INFOR==ITEM) { LOC=START; LOCP=null; return true;}

•
•
•     SAVE=START; PTR=START.LINK;
•     while(PTR!=null){
•         if((int)PTR.INFOR==ITEM) {
•             LOC = PTR; LOCP=SAVE;
•             return true;
•         }
•         SAVE=PTR;
•         PTR=PTR.LINK;
•     }
•     LOC=null;
•     //Show Result;
•     if(LOC ==null){
•         System.out.println("Not found.");
•         return false;
•     }
•     else
•     {
•

```

- public class Algor59to10TFINDB {
- static Node START = new Node();
- static Node LOCP=null;
- static Node LOC=null;
- public static void main(String[] args) {
- START.INFOR = 50;
- Node A = new Node(); A.INFOR = 60; START.LINK = A;
- Node B = new Node(); B.INFOR = 70; A.LINK = B;
- Node C = new Node(); C.INFOR = 80; B.LINK = C;
- Node D = new Node(); D.INFOR = 90; C.LINK = D;
- Node E = new Node(); E.INFOR = 100; D.LINK =E;
- System.out.println("Data Befor INSERT:");
- showDatainList();
- DEL(70); //Call INSLOC()...
- System.out.println("Data After INSERT:");
- showDatainList();
- }



- `public static void DEL(int ITEM){`
- `//Algorrithm 5.8 Delete`
- `FINDB(ITEM);`
- `if(LOCP==null) {`
- `START = START.LINK; //insert first Node`
- `}else`
- `{`
- `LOCP.LINK = LOC.LINK;`
- `}`
- `}`

```

• public static boolean FINDB(int ITEM){
•
•     Node PTR=null;
•     Node SAVE=null;
•     PTR=START;
•     if(START==null) { LOC=null; LOCP=null; return true;}
•     if((int)START.INFOR==ITEM) { LOC=START; LOCP=null; return true;}
•
•
•
•     SAVE=START; PTR=START.LINK;
•     while(PTR!=null){
•         if((int)PTR.INFOR==ITEM) {
•             LOC = PTR; LOCP=SAVE;
•             return true;
•         }
•         SAVE=PTR;
•         PTR=PTR.LINK;
•     }
•     LOC=null;
•     //Show Result;
•     if(LOC ==null){
•         System.out.println("Not found.");
•         return false;
•     }
•     else
•     {
•         System.out.println("Found.");
•         return true;
•     }
• }

```

กิจกรรมนักศึกษา

- รันโปรแกรมทดสอบการทำงานของอัลกอริทึม
- ทดลองเขียนโปรแกรมตามที่โจทย์กำหนด

- 
- จบส่วนที่ 1 ต่อส่วนที่ 2